School of Computer Science
FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

Transdermal Drug Diffusion: An End-to-End Finite-Difference Simulation and Physics-Corrected Surrogate Framework

Nathan Walsh

Submitted in accordance with the requirements for the degree of BSc Computer Science with Artificial Intelligence

2025/2026

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (27/04/26)
Link to online code repository	Github Repository URL	Sent to supervisor and assessor (24/04/26)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Nathan Walsh

Summary

Predicting drug permeation through skin remains computationally challenging despite the clinical importance of transdermal delivery as a non-invasive therapeutic route. One method is mechanistic simulation of drug diffusion through layered skin tissue, though this becomes computationally expensive across large parameter configurations. Recent developments in machine learning have made surrogate models a practical way to approximate such simulations at a fraction of the compute cost. Purely data-driven surrogates excel in efficiency but have no inherent respect for the governing transport physics, and may produce physically implausible predictions outside the training distribution. This project addresses this problem with an end-to-end computational framework comprising a progressively verified simulator, a black-box surrogate model, and a physics-corrected hybrid, evaluating whether the physics-corrected hybrid improves on the black-box baseline while preserving practical computational advantages.

The simulator was developed progressively in three regimes. V1 was a homogeneous baseline verified through grid-refinement convergence analysis and comparison against a one-dimensional analytic benchmark. V2 introduced layered transport coefficients calibrated against published lidocaine hydrochloride permeability and lag-time data, reproducing both targets to within 0.1%. V3 built on these as a finite-dose heterogeneous regime, generating 1,000 simulation runs spanning patch geometry, donor conditions, dermal clearance, and spatial diffusivity heterogeneity.

The black-box baseline applies principal component analysis to log-transformed flux curves followed by Ridge regression. The physics-corrected hybrid builds on this with a bounded additive correction, using a validation-selected blending step to reduce degradation where the correction was less reliable. Both models were evaluated on a held-out test set using flux curve-level metrics, derived scalar transport quantities, physics-based diagnostics, and runtime comparisons, with robustness assessed across multiple random seeds.

The black-box baseline achieved strong flux curve prediction performance on the held-out test set (relative L_2 error 0.0789, Pearson correlation 0.9967), confirming the V3 dataset supports meaningful surrogate learning. The physics-corrected hybrid surrogate produced further improvements in curve accuracy and scalar transport metrics, reducing the relative L_2 error from 0.0789 to a mean of approximately 0.0505 across three independent random seeds, with the validation-selected blending strategy limiting degradation where the correction was less reliable. Surrogate inference for both models is several orders of magnitude faster than the numerical simulator ($\sim$2,800,000$\times$ for the black-box and $\sim$12,200$\times$ for the hybrid on CPU; $\sim$414,000$\times$ for the hybrid on GPU), supporting practical utility for parameter studies. Robustness analysis across multiple seeds confirms these improvements are consistent, establishing that a physics-corrected hybrid surrogate, grounded in a simulator framework validated through progressive verification and calibration, offers credible and computationally practical performance within the sampled V3 parameter space.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Timon Gutleb, for his guidance, feedback, and support throughout this project, particularly in helping me overcome the initial hardware limitations. His support was invaluable in enabling the progress of this work. I am also grateful to my coursemates over the past three years, who have deepened my enthusiasm for computer science and continually motivated me to challenge myself in a friendly and competitive environment, while also becoming lifelong friends. Finally, I would like to thank my family and friends for their continued encouragement and support throughout my degree. I would not be in the fortunate position I am in today without them.

Contents

1 Introduction and Background Research **1**

1.1 Introduction . . . 1

1.1.1 Motivation and Biomedical Context . . . 1

1.1.2 Computational Context and Problem Statement . . . 2

1.1.3 Aim, Objectives, and Research Questions . . . 2

1.1.4 Project Scope and Contributions . . . 3

1.2 Background Research and Literature Review . . . 4

1.2.1 Mathematical and Biomedical Modelling of Skin Diffusion . . . 4

1.2.2 Numerical Methods for Diffusion Problems . . . 4

1.2.3 Data-Driven Surrogate Modelling . . . 5

1.2.4 Physics-Corrected Surrogate Modelling . . . 5

1.2.5 Summary of Research Gap and Project Positioning . . . 6

2 Methodology **7**

2.1 End-to-End Framework Overview . . . 7

2.2 Physical Model, Governing Equations, and Outputs . . . 7

2.2.1 Governing Transport Equation . . . 8

2.2.2 Initial and Boundary Conditions . . . 8

2.2.3 Transport Outputs and Derived Scalar Quantities . . . 9

2.3 Progressive Simulator Design: `V1`, `V2`, and `V3` . . . 9

2.3.1 `V1`: Homogeneous Baseline for Numerical Verification . . . 10

2.3.2 `V2`: Layered Literature-Anchored Model . . . 10

2.3.3 `V3`: Final Heterogeneous Finite-Dose Regime for Dataset Generation . . . 10

2.4 Numerical Method and Stability . . . 11

2.5 Dataset Design and Sampling Strategy . . . 11

2.5.1 Parameter Space Definition . . . 11

2.5.2 Dataset Assembly and Split Policy . . . 12

2.5.3 Feature Engineering . . . 12

2.6 Surrogate Modelling Methodology . . . 12

2.6.1 Black-Box Baseline . . . 12

2.6.2 Physics-Corrected Hybrid Surrogate . . . 13

2.6.3 Multi-Seed Training and Robustness 14
2.7 Evaluation Metrics and Success Criteria 14
2.7.1 Curve-Level Metrics 14
2.7.2 Scalar Metrics 14
2.7.3 Physics Diagnostics 14
2.7.4 Runtime and Practical Utility 15
2.8 Reproducibility and Quality Assurance 15

3 Implementation and Validation 16
3.1 Software Architecture and Repository Organisation 16
3.2 Simulator Implementation 16
3.2.1 Grid, Fields, and Boundary Handling 16
3.2.2 Metrics and Run Artefacts 17
3.2.3 Simulator Regimes 17
3.3 Validation of the Simulator 17
3.3.1 `V1` Grid-Convergence Verification 17
3.3.2 `V1` One-Dimensional Analytic Benchmark 18
3.3.3 `V2` Literature Calibration and Validation 19
3.3.4 Justification of `V3` as the Final Simulator Regime 19
3.4 Dataset Pipeline Implementation 20
3.4.1 Run Generation and Integrity Checking 20
3.4.2 Assembly, Export and Quality Control 21
3.5 Surrogate Pipeline Implementation 21
3.5.1 Black-Box Baseline Implementation 21
3.5.2 Physics-Corrected Hybrid Implementation 22
3.5.3 Diagnostics, Logging, and Output Artefacts 22
3.6 Testing and Reproducibility 22

4 Results, Evaluation, and Discussion 23
4.1 Evaluation Basis and Dataset Readiness 23
4.2 Black-Box Baseline Results 23
4.3 Physics-Corrected Hybrid Results 24
4.4 Comparative Evaluation of the Two Surrogate Strategies 25
4.5 Training Stability and Computational Cost 28
4.5.1 Multi-Seed Stability of the Hybrid Results 28
4.5.2 Computational Cost and Practical Trade-Offs 29

4.6 Discussion in Relation to the Project Objectives 29
4.6.1 Limitations and Future Work 30

References **31**

Appendices **34**

A Self-appraisal **34**
A.1 Critical self-evaluation 34
A.2 Personal reflection and lessons learned 35
A.3 Legal, social, ethical and professional issues 36
A.3.1 Legal issues 36
A.3.2 Social issues 36
A.3.3 Ethical issues 37
A.3.4 Professional issues 37

B External Material **38**

C Discretisation of the Bottom Flux **39**

D Training Objective: Component Loss Definitions **40**

E Evaluation Metric Definitions **42**

F Physics Diagnostic Definitions **43**

G 1D Analytic Benchmark Derivation **44**

H Repository Layout **46**

I Dataset Quality Control Plots **47**

J Unit Test Modules **48**

K Scalar Transport Quantity Definitions **49**

L Hybrid Surrogate Training Convergence **50**

Chapter 1

Introduction and Background Research

1.1 Introduction

1.1.1 Motivation and Biomedical Context

In clinical practice, transdermal drug delivery is a well-established, non-invasive route for administering medication and therapeutic agents. Common applications include anaesthetics such as lidocaine, nicotine replacement and hormone therapy [1]. It has advantages over oral drug administration in that it avoids hepatic first-pass metabolism, which is the breakdown of medication by the liver before it reaches systemic circulation. Furthermore, it enables sustained drug release over extended periods, improving tolerability and patient adherence [1].

Drug transport through skin is predominantly driven by passive diffusion towards systemic circulation [1]. This is described by Fickian diffusion, which relates the diffusive flux (the rate of drug movement per unit area) to the local concentration gradient [2]. The stratum corneum, viable epidermis, and dermis are the three main layers of human skin that are commonly modelled [3]. Of these, the stratum corneum generally provides the dominant resistance to permeation (the passage of drug through the skin layers) [1, 3]. However, diffusion through the deeper skin layers and clearance into the dermal blood vessels can also affect permeation by altering the concentration gradient and sink conditions (zero drug concentration at the dermis base, modelling clearance into the bloodstream) [3], as illustrated in Figure 1.1.

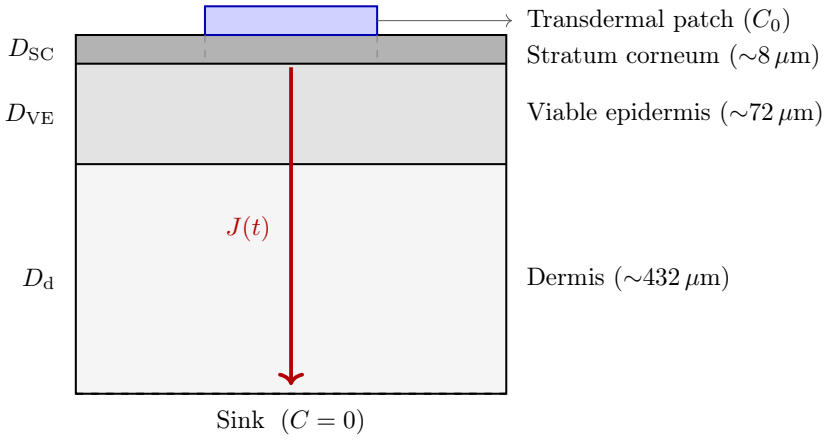


Figure 1.1: Three-layer skin model with transdermal patch (C_0) at the top boundary and sink condition ($C = 0$) at the base. Drug diffuses down the concentration gradient from patch to sink, where $J(t)$ (red arrow) is the transdermal flux at the lower boundary.

Given this layered complexity, quantitative multilayer diffusion modelling offers a useful framework for transdermal delivery system research and design. This enables the prediction of clinically relevant quantities such as flux, permeability, and lag time [3]. The transient flux profile $J(t)$ is a measure of drug permeation over time and is the primary quantity of interest in this work, from which a range of scalar metrics are subsequently derived (see Appendix K).

1.1.2 Computational Context and Problem Statement

Mechanistic diffusion models solve governing transport equations directly over the skin domain and are physically informative, but remain computationally expensive to evaluate frequently across a wide range of parameter configurations. Empirical models, such as that of Potts and Guy [4], provide rapid estimates of skin permeability from molecular properties, but they do not resolve how concentration varies across the skin over time or accommodate more complex boundary behaviour. Numerical solvers offer greater mechanistic detail but can become computationally demanding when applied across large parameter studies, a challenge that is well established in scientific simulation more broadly [5].

The availability of scalable neural network training and large synthetic datasets has created practical opportunities for surrogate acceleration. Once trained, a surrogate model can approximate the behaviour of numerical solvers at a fraction of the computational cost [5]. However, purely data-driven ("black-box") models do not inherently respect governing physics or conservation behaviour, and as a result may produce physically implausible predictions outside the range of inputs seen during training [6]. A complementary approach is to augment a data-driven baseline with a physics-aware corrective component, preserving surrogate speed while improving the degree to which predictions conform to the governing transport physics. These composite models have shown promise in improving accuracy and physical consistency relative to purely data-driven baselines [7, 8].

This project therefore evaluates surrogate acceleration of transdermal diffusion simulation, comparing a black-box surrogate baseline against a physics-corrected hybrid refinement within a mechanistically grounded and progressively verified end-to-end framework.

1.1.3 Aim, Objectives, and Research Questions

Aim

To develop an end-to-end computational framework for transdermal drug diffusion, centred on a progressively verified, literature-calibrated finite-difference simulator. Using this simulator as the data source, the project evaluates whether a physics-corrected hybrid surrogate improves upon a purely data-driven baseline while preserving practical computational advantages.

Objectives

To achieve this aim, the project is structured around the following objectives:

- Implement a finite-difference numerical solver for transdermal diffusion, progressing from simplified settings to more realistic configurations.
- Validate the simulator through numerical verification, analytic benchmarking, and calibration against published literature.
- Build a structured pipeline for dataset generation for surrogate learning.
- Design and train a black-box surrogate model as a strong reference baseline.
- Develop a physics-corrected hybrid surrogate that refines the black-box prediction.
- Evaluate both models using curve-level, scalar transport, physics-based, and runtime metrics, with robustness analysis across multiple random seeds.

Research Questions

The project addresses the following research questions:

- How effectively can a finite-difference transdermal diffusion simulator be validated progressively from numerical verification through to literature-anchored calibration, and what evidence does each stage provide?
- Does a physics-corrected hybrid surrogate improve upon a purely data-driven baseline in predicting flux curves and derived scalar transport metrics, and are any such improvements stable across multiple training runs?
- What trade-offs arise between predictive performance and computational cost when comparing the black-box and physics-corrected surrogate approaches?

1.1.4 Project Scope and Contributions

Scope and Boundaries

This project focuses on a computational model of layered transdermal diffusion, rather than a pharmacokinetic description of drug behaviour within the body. The model represents transport in both depth and lateral directions, capturing layered structure, patch geometry, donor behaviour, and spatial heterogeneity at an appropriate resolution for controlled simulation and surrogate experimentation. Published literature data is used to anchor calibration for macroscopic outputs rather than to supply layer-specific transport coefficients directly. The work does not pursue patient-specific prediction, clinical validation, or experimental data collection, as these lie beyond the scope of framework comparison.

Contributions

Simulator A finite-difference diffusion simulator developed progressively from a simple homogeneous baseline to a layered, literature-anchored model and, subsequently, to a heterogeneous finite-dose configuration used for final data generation.

Validation A validation framework comprising grid-convergence analysis, a one-dimensional analytic benchmark, and literature-based calibration to permeability and lag-time targets.

Dataset pipeline A structured data-generation, assembly, export, and quality-control pipeline derived from the final verified-and-calibrated simulator configuration.

Surrogate models A black-box baseline model and a physics-corrected hybrid surrogate designed to refine baseline predictions through additive physics corrections.

Evaluation A comparative evaluation framework incorporating curve-level metrics, scalar transport metrics, robustness analyses, and runtime comparisons.

Reproducibility A configuration-driven, version-controlled workflow supporting reproducible simulation, dataset generation, and model evaluation, which together form an end-to-end framework from physical simulation to evaluated surrogate modelling.

1.2 Background Research and Literature Review

1.2.1 Mathematical and Biomedical Modelling of Skin Diffusion

Mathematical modelling of transdermal drug transport is typically based on classical diffusion theory, with Fick's second law describing concentration dynamics [2]. Such models are commonly extended to feature layered anatomical structure, spatially varying material properties, and clearance processes [9]. These representations are key to capturing realistic permeation behaviour [1, 3].

Empirical models, most notably that of Potts and Guy [4], relate skin permeability to physicochemical molecular properties and provide rapid estimates of transdermal transport, but cannot resolve the transient concentration fields and flux trajectories required for simulator validation and surrogate learning. Earlier membrane-based approaches treated the skin as a set of discrete diffusion-limited layers and allowed closed-form solutions at the expense of spatially resolved concentration fields [10]. Mechanistic models address this limitation by explicitly representing layered transport, offering greater physical detail and interpretability at the cost of computational complexity [11, 9]. Obtaining such spatially resolved solutions from mechanistic models requires numerical discretisation of the governing equations, typically via finite-difference or finite-element methods.

Spatially resolved concentration fields and flux trajectories are needed for validation, surrogate training, and transport metric evaluation; this motivates the use of a mechanistic simulator in this work. Where direct layer-specific coefficients are unavailable, calibration against macroscopic quantities such as permeability and lag time remains a practical modelling strategy [9, 11].

1.2.2 Numerical Methods for Diffusion Problems

Closed-form solutions to diffusion equations exist only for idealised, homogeneous settings [2]. More realistic transdermal models involving layered media, heterogeneous coefficients or finite-dose behaviour must be solved numerically. Finite-difference solvers are often used because they are simple to implement and closely follow the structure of the partial differential equation (PDE), preserving a direct link between the mathematical model and its numerical approximation [2, 12]. However, stability of these schemes requires careful time-step control, and numerical accuracy must be confirmed through appropriate verification procedures. By contrast, finite-element methods offer greater geometric flexibility but this comes with higher implementation complexity [12]. Such trade-offs mean that finite-difference schemes are a more natural choice where interpretability and a stable computational baseline take priority, as in the present work. Since the surrogate models are trained directly on simulator-generated data, data credibility affects the quality of the learned model. This makes verification an essential prerequisite for surrogate learning, comprising three stages: grid-convergence analysis (progressively refining the mesh to confirm solution convergence and controlled discretisation error); analytic benchmarking against known closed-form solutions; and calibration against literature-derived transport quantities.

1.2.3 Data-Driven Surrogate Modelling

Machine learning surrogates are increasingly adopted as fast emulators of numerical simulation [13]. Once trained, the surrogate can often be evaluated much faster than the underlying numerical solver, making it well-suited for parameter sweeps, optimisation, uncertainty analysis and real-time prediction [5]. In practice, the suitability of a surrogate depends not only on its predictive accuracy but also on its ability to scale to large synthetic datasets and structured outputs such as full diffusion profiles [6]. This is particularly relevant here as the nature of the task is not just to predict scalar quantities but predominantly time-dependent transport behaviour.

Classical surrogate families such as Gaussian process regression are well established for scalar outputs and moderate dataset sizes [13]. However, they become impractical and scale poorly for larger datasets due to their cubic training complexity of $O(n^3)$ where n is the number of training points [14]; therefore, they are not naturally suited to high-dimensional structured outputs such as time-series flux curves. Although sparse approximations can reduce this cost, they introduce additional modelling choices and remain less natural for structured time-series outputs [14]. A more tractable option is to first compress the output space through dimensionality reduction and then learn a mapping in the reduced representation. This strategy preserves interpretability while remaining computationally feasible at the relevant scales. Fully connected neural networks on raw time-series data provide an alternative but offer less interpretability and require more careful tuning, making validation against physical quantities of interest harder [6]. The surrogate baseline in this work therefore adopts this two-stage strategy: principal-component compression of the flux output followed by a learned regression in the reduced representation.

Despite these computational advantages, black-box surrogates introduce important modelling trade-offs. Their performance is strongly dependent on the quality and coverage of the training data, and they may struggle to generalise outside the sampled parameter range. More importantly, physical constraints such as conservation of mass, prescribed boundary conditions (e.g. the fixed donor concentration at the patch surface), and admissibility conditions such as non-negative concentration values are not explicitly encoded in black-box models [6]. As a result, low error on held-out validation data does not necessarily guarantee physically credible behaviour. For this reason, data-driven surrogates are treated here as strong computational baselines rather than complete replacements for mechanistic modelling.

1.2.4 Physics-Corrected Surrogate Modelling

Embedding physical structure into learned surrogates can improve both predictive accuracy and generalisation, particularly in regions of parameter space under-represented in the training data. Two main approaches exist: physics-informed neural networks (PINNs) and hybrid corrective models. PINNs embed physical laws directly into the training objective via governing-equation residuals, boundary conditions, and initial conditions [15]. While this can improve physical consistency and generalisation relative to unconstrained black-box models [6], PINN training is sensitive to loss-term weighting (the relative importance of the PDE, boundary, and initial condition terms in the training objective) and can become unstable, often requiring careful problem-specific tuning to match well-regularised data-driven baselines [16, 6].

Given the priority of maintaining a stable baseline and controlled comparison, a more tractable alternative is to embed physical structure selectively. This involves building a physics-aware corrective component on top of a stable data-driven baseline rather than requiring the learned model to represent the full solution directly. In scientific machine learning, hybrid methods of this kind allow learned components to augment rather than replace the underlying physical description [17, 8]. Formally, this takes the general form $\hat{y} = f_{\text{base}}(x) + \delta(x)$, where f_{base} is the data-driven baseline and $\delta(x)$ is a learned corrective term - a structure rooted in the model discrepancy framework of Kennedy and O'Hagan [18]. The correction learns the residual discrepancy between the baseline prediction and the true output, with the composite model shown to substantially outperform purely learned approaches, particularly where training data is limited [7, 8]. This design principle sidesteps loss-weighting sensitivity while preserving links to the governing transport behaviour, and is the approach adopted in the present work.

1.2.5 Summary of Research Gap and Project Positioning

The literature reviewed above shows that the component areas relevant to this project are each individually well developed. Biomedical and mathematical studies of skin diffusion provide a strong mechanistic basis for modelling transdermal transport [1, 3]. Numerical methods provide the tools needed to solve these models in settings where analytic solutions are unavailable [2]. Data-driven surrogate models offer substantial potential for acceleration [5], while physics-corrected hybrid approaches that augment a stable baseline with a learned corrective component have been shown to outperform purely data-driven models while better preserving physical fidelity [7, 8]. However, these demonstrations primarily target generic PDE benchmarks rather than domain-specific structured outputs such as time-series transport profiles, leaving their practical advantage in applied diffusion settings unexplored.

What is less evident in the literature considered for this project is a single end-to-end framework unifying simulator development, validation, surrogate-data generation, and surrogate comparison for transdermal diffusion. In particular, the literature does not clearly show a workflow in which a progressively verified, literature-anchored mechanistic simulator is used to generate training data. It also does not clearly show a controlled comparison between a purely data-driven surrogate and a physics-corrected hybrid alternative using transport-relevant metrics. Existing discussions of surrogate performance are often presented mainly in terms of generic prediction error, whereas in this application it is also important to assess quantities such as flux, permeability, lag time, physical consistency, robustness, and computational cost.

The present project addresses this gap by developing a controlled computational framework for layered transdermal diffusion and establishing its credibility through numerical verification, analytic benchmarking, and literature-anchored calibration. It then uses this framework to carry out a structured comparison between black-box and physics-corrected hybrid surrogate modelling. The contribution is therefore not only a comparison of surrogate strategies, but an end-to-end methodology in which surrogate evaluation is explicitly grounded in a credible mechanistic simulator and interpreted through domain-relevant transport metrics.

Chapter 2

Methodology

2.1 End-to-End Framework Overview

Rather than treating the project as having separate simulation and machine learning tasks, the methodology is structured as a single end-to-end computational workflow, summarised in Figure 2.1. The finite-difference simulator comes first, developed through increasing complexity to model layered transdermal drug transport. This progression is verified first through numerical checks on a simpler configuration and then through literature-anchored calibration in a more realistic setup. The final `V3` regime, built on the verified `V1` and literature-anchored `V2` stages, is used to generate a credible dataset upon which the surrogate models depend. A black-box surrogate is first trained as a baseline for predicting the flux curve $J(t)$, and a physics-corrected hybrid is then built on top of it. Both are evaluated using curve-level metrics, scalar transport quantities, physics diagnostics, robustness checks, and runtime analysis.

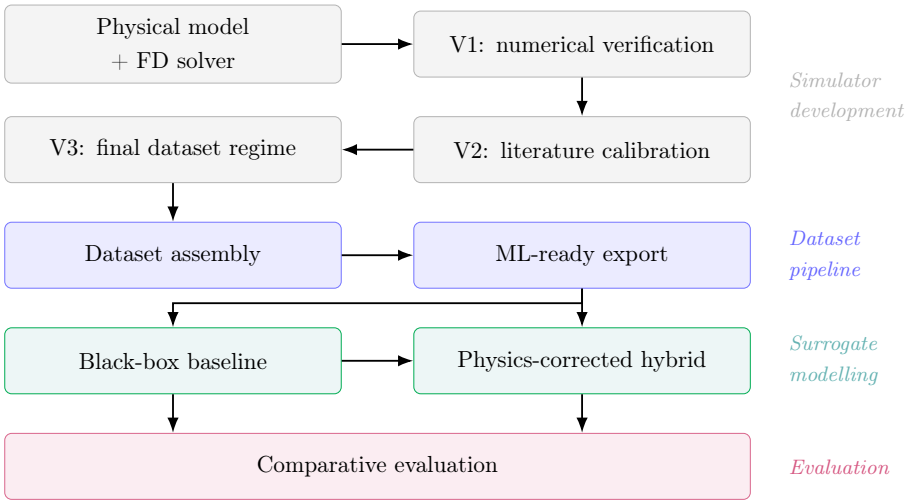


Figure 2.1: Overview of the end-to-end computational framework.

2.2 Physical Model, Governing Equations, and Outputs

Transdermal transport is represented using a two-dimensional vertical cross-section of skin. The computational domain is defined as

$$\Omega = [0, L_z] \times [0, L_x], \qquad \begin{array}{ll} \text{where } z : \text{depth coordinate} & L_z : \text{total skin thickness} \\ x : \text{lateral coordinate} & L_x : \text{lateral skin length} \end{array} \tag{2.1}$$

The model is intended as an effective transport description, offering a controlled mechanistic setting that is sufficiently rich for validation and surrogate-learning experiments, rather than a full physiological representation of transdermal drug uptake.

Following standard mechanistic skin-permeation modelling, drug transport is assumed to be dominated by passive diffusion in response to concentration gradients [1, 9]. Active transport, explicit metabolism, and detailed vehicle-skin interactions are omitted as the aim is to construct a simulator that remains tractable, interpretable, and suitable for controlled surrogate-model comparison [9].

Skin is represented through layers rather than microscopic anatomy. The domain is split into stratum corneum, viable epidermis, and dermis. Each layer is assigned its own effective diffusion coefficient, D, which controls how rapidly drug diffuses within that layer, with no explicit partition coefficients introduced between layers. Layer transitions are instead captured through step changes in diffusivity; the conservative discretisation ensures flux-consistent transport throughout [12].

Dermal clearance is modelled as a first-order sink in the dermal region, consistent with vascular removal occurring in deeper tissue [11]. A perfect-sink boundary condition at the domain base represents systemic removal of drug that reaches the bloodstream.

2.2.1 Governing Transport Equation

Let $C(z, x, t)$ denote drug concentration within the skin domain. The simulator is based on the diffusion-reaction equation within a standard Fickian modelling framework [2].

$$\frac{\partial C}{\partial t} = \underbrace{\nabla \cdot (D(z,x)\,\nabla C)}_{\text{Fickian diffusion}} - \underbrace{k(z,x)\,C}_{\text{first-order clearance}} \tag{2.2}$$

The diffusion term represents passive transport driven by concentration gradients, while the reaction term represents first-order loss, where $k > 0$ in the dermal region and $k = 0$ elsewhere [11]. For diffusion, the conservative form $\nabla \cdot (D\nabla C)$ is adopted for spatially varying and discontinuous diffusivity across layered and heterogeneous regions.

2.2.2 Initial and Boundary Conditions

Boundary conditions are determined by patch geometry and donor concentration $C_d(t)$:

$$\begin{aligned}
C(z,x,0) &= 0 && \text{[initial: skin assumed drug-free at } t = 0\text{]} \\
C(0,x,t) &= C_d(t) && \text{[top (on-patch): prescribed donor concentration]} \\
\left.\frac{\partial C}{\partial z}\right|_{z=0} &= 0 && \text{[top (off-patch): no drug enters outside patch area]} \\
C(L_z,x,t) &= 0 && \text{[bottom: perfect sink for vascular removal]} \\
\left.\frac{\partial C}{\partial x}\right|_{x=0,\,L_x} &= 0 && \text{[lateral: domain is sealed, no drug exits sideways]}
\end{aligned} \tag{2.3}$$

where the on-patch donor concentration is:

$$C_d(t) = \begin{cases} C_0 & \text{[infinite dose patch: constant donor supply]} \\ C_0\, e^{-\lambda t} & \text{[finite-dose patch: exponential donor depletion]} \end{cases} \tag{2.4}$$

where $\lambda > 0$ is the donor decay rate. The finite-dose proxy is consistent with diffusion-based modelling of finite vehicle volume in transdermal absorption [19].

2.2.3 Transport Outputs and Derived Scalar Quantities

The primary output of the simulator is the bottom flux (rate at which drug crosses into the sink boundary), whose continuous form follows from Fick's first law [2]; the discrete approximation is obtained by finite difference and lateral averaging (derivation in Appendix C):

$$J(t) = -D\left.\frac{\partial C}{\partial z}\right|_{z=L_z} \qquad J(t_n) \approx -\frac{1}{W}\sum_{j=1}^{W} D_{H-1,j}\frac{C_{H,j}^{(n)} - C_{H-1,j}^{(n)}}{\Delta z} \tag{2.5}$$

where H is the sink boundary row index, W the lateral grid width, Δz the uniform grid spacing, and the discrete form averages the gradient between rows $H-1$ and H across all W columns. This quantity is the central target for simulator analysis and surrogate modelling.

Several scalar quantities are derived from $J(t)$, grouped by regime since steady-state measures are only meaningful under infinite-dose conditions, due to the nature of its flux curve:

$$\begin{aligned} &\text{V1–V2:} \quad J_{\text{ss}} = \frac{1}{N_{\text{tail}}}\sum_{n\in\text{tail}} J(t_n), \quad P = \frac{J_{\text{ss}}}{C_0}, \quad T_{\text{lag}} : Q(t) \approx J_{\text{ss}}(t - T_{\text{lag}}) \text{ (late-time linear fit)} \\ &\text{V3:} \qquad J_{\text{peak}} = \max_t J(t), \quad t_{\text{peak}} = \arg\max_t J(t), \quad M_{\text{delivered},24h} = \int_0^{24h} J(t)\,dt \end{aligned} \tag{2.6}$$

For `V3`, where a finite-donor patch is used, donor depletion over time prevents a true steady-state from being reached, so J_{ss} and P lose physical interpretability; the more meaningful evaluation quantities are peak and cumulative delivery metrics. Here J_{peak} captures the maximum flux reached during delivery, t_{peak} the time at which it occurs, and $M_{\text{delivered},24h}$ the total drug mass delivered over a 24-hour window. Together these characterise the onset, intensity, and duration of the delivery event. The geometric relationship between each scalar and the flux curve is illustrated in Appendix K; the `V1`–`V3` regimes are defined below.

2.3 Progressive Simulator Design: `V1`, `V2`, and `V3`

The simulator was constructed progressively in three regimes, each with distinct methodological roles supporting verification, validation and eventual dataset generation for surrogate modelling. Figure 2.2 and Figure 2.3 demonstrate the key differences between the three versions in terms of their diffusivity fields and resulting flux curves.

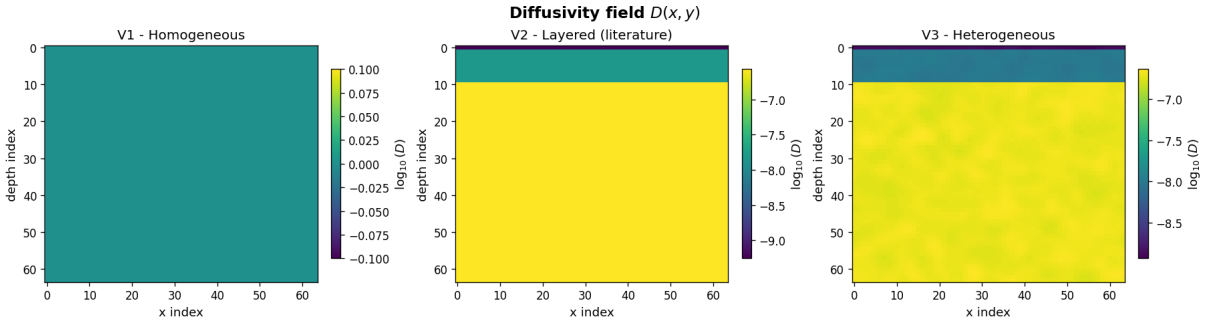


Figure 2.2: Representative diffusivity field $D(z, x)$ on a $\log_{10}$ scale for each simulator regime. **V1** (left): spatially uniform field used for numerical verification. **V2** (centre): three-layer structure calibrated to reproduce literature-reported permeability and lag-time behaviour for lidocaine hydrochloride [20]. **V3** (right): layer-preserving spatial heterogeneity introduced at cell level within the final dataset-generation regime.

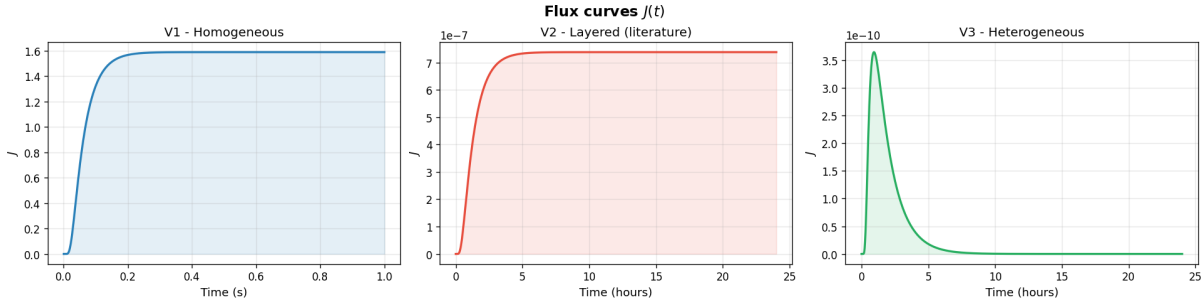


Figure 2.3: Representative bottom-boundary flux curve $J(t)$ for each simulator regime. **V1** (left): monotonic rise to steady state. **V2** (centre): pronounced lag phase before plateau, reflecting the stratum corneum barrier in the layered literature-calibrated regime [20]. **V3** (right): rise-peak-fall behaviour under the finite-dose proxy, yielding a richer temporal prediction target for surrogate modelling.

2.3.1 `V1`: Homogeneous Baseline for Numerical Verification

`V1` is the simplest simulator regime featuring a homogeneous diffusion field, no dermal clearance and a full-width infinite-dose patch. The purpose is to establish the trustworthiness of the numerical solver. Its homogeneous structure makes it suitable for grid-refinement tests and comparison against an idealised one-dimensional analytic benchmark [2]. This regime verifies that the finite-difference solver converges consistently under grid refinement and reproduces expected diffusion behaviour.

2.3.2 `V2`: Layered Literature-Anchored Model

`V2` introduces anatomically interpretable layering with the stratum corneum, viable epidermis, and dermis. The domain and simulation timescale are matched to the literature setup, with layer-wise diffusivities calibrated to reproduce reported permeability and lag-time behaviour for lidocaine hydrochloride [20]. Its role is to bridge mathematical correctness and pharmacokinetic realism, demonstrating that the solver can reproduce macroscopic permeation behaviour reported in the literature. The calibration targets are permeability P and lag time T_{lag}, since direct layer-resolved coefficients are rarely available in the literature. The aim is not to recover a unique microscopic ground truth but instead to reproduce realistic transdermal behaviour in the outputs most relevant to validation.

2.3.3 `V3`: Final Heterogeneous Finite-Dose Regime for Dataset Generation

`V3` is the final simulator regime, constructed for dataset generation. It keeps the layered structure and assumptions established in `V1` and `V2`, but introduces a finite-dose donor patch, partial patch coverage, spatial diffusion heterogeneity [9] and dermal clearance. This creates a rise-peak-fall flux curve (Figure 2.3), rather than the previous steady-state responses. This curve motivates a surrogate learning problem that is both credible and sufficiently challenging to support meaningful evaluation of predictive performance, robustness, and physics-aware correction.

2.4 Numerical Method and Stability

The simulator discretises the governing equation using an explicit finite-difference scheme [12]:

$$C^{n+1} = C^n + \Delta t \left[\nabla \cdot (D \nabla C) - kC\right]^n . \tag{2.7}$$

The computational grid is fixed across all regimes at $H \times W = 64 \times 64$ cells, balancing spatial resolution against dataset generation time while remaining stable at the diffusivity values used. In the layered `V2` and `V3` regimes, this gives $\Delta z = \Delta x \approx 8\,\mu$m per cell, the stratum corneum occupying one row, the viable epidermis nine, and the dermis the remaining fifty-four. The `V1` regime uses the same grid but with a uniform diffusivity field and no layer partitioning. The single-row SC in `V2` and `V3` is a known resolution limitation. However, its barrier role is preserved through D_{SC}, which is several orders of magnitude below the deeper layers and is consistent with representations used in discrete-layer transdermal models [9, 3]. Alternative schemes such as Crank-Nicolson remove the stability constraint but introduce a linear solve at each step, which does not improve accuracy at the grid resolutions used here [12].

Varying diffusivity (e.g. `V2` and `V3` regimes) is handled via harmonic-mean face values [21], a standard approach for discontinuous coefficients in layered media. The time step must satisfy:

$$D_{\mathrm{face}} = \frac{2 D_i D_j}{D_i + D_j} \qquad \Delta t \leq \min\left(\frac{h^2}{4 D_{\max}},\ \frac{1}{k_{\max}}\right) \tag{2.8}$$

where D_i, D_j are diffusivities at adjacent cells, h is the uniform grid spacing, $k_{\max}$ is the maximum dermal clearance rate, and the second term applies only when dermal clearance is active, ensuring the explicit scheme remains stable under first-order reaction.

2.5 Dataset Design and Sampling Strategy

2.5.1 Parameter Space Definition

The `V3` dataset samples over input variables chosen to influence the delivered flux curve in physically meaningful ways, covering patch geometry, donor conditions, dermal clearance, and diffusion-field heterogeneity (Table 2.1). These choices directly alter the flux curve at the sink boundary and ensure the problem is both challenging and representative.

Parameter	Type	Range / Values	Notes
`patch_width`	Discrete	$\{0.25,\ 0.5,\ 1.0\}$	Fraction of domain width
`patch_offset`	Discrete	left, centre, right	Forced centre when width = 1.0
C_0	Uniform	$[0.8,\ 1.2]$	Donor concentration
`decay_rate` λ	Uniform	$[0.05,\ 0.3]\,\mathrm{s}^{-1}$	Finite-dose depletion rate
k_{dermis}	Discrete	$\{0,\ 10^{-5},\ 3.3{\times}10^{-5},\ 10^{-4}\}\,\mathrm{s}^{-1}$	Dermal clearance strength
`het_sigma`[a]	Uniform	$[0.01,\ 0.08]$	Log-space variability of D field
`het_steps`[a]	Random integer	$[3,\ 9]$	Correlation length of D field

[a] Abbreviated from `heterogeneity_sigma` and `heterogeneity_steps`.

Table 2.1: Sampled parameter space for the `V3` dataset (20 features total after engineering). Stratification uses `patch_width` and `patch_offset` to prevent leakage across splits.

2.5.2 Dataset Assembly and Split Policy

The dataset pipeline proceeds in stages. Runs are first generated from the sampled parameter configurations then checked for integrity. If they are valid they are assembled into train, validation and test sets and exported into a machine-learning-ready form. Each run stores the sampled inputs, flux curve, derived scalar metrics, time grid, and the realised diffusivity field.

The 1,000-run dataset is split into three subsets: training 70%, validation 15%, and test 15%. This uses a deterministic fixed-seed policy with stratification on input-side labels rather than output targets, keeping the split leakage-free while preserving balance across the data.

2.5.3 Feature Engineering

The machine-learning export combines sampled inputs with engineered features representing meaningful interactions and log-transformed parameters into a 20-dimensional vector.

Feature group	**Features**	***N***
Direct sampled inputs	`patch_width`, `patch_offset`[a], C_0, `decay_rate`, k_{dermis}, `heterogeneity_sigma`, `heterogeneity_steps`	7
Engineered scaling and interaction terms	`dose_proxy_c0_over_decay`, log(`decay_rate`), `width_times_sigma`, `c0_times_width`, `decay_times_width`, `sigma_times_width`	6
Diffusion-field summary statistics	`D_mean`, `D_std`, `D_p10`, `D_p50`, `D_p90`, `D_top_mean`, `D_bottom_mean`	7
Total		**20**

[a] `patch_offset` is encoded numerically from the categorical left/centre/right patch-position setting.

Table 2.2: Feature groups used in the exported machine-learning dataset (20 features total).

This feature design keeps the surrogate input space manageable while allowing both models to account for the realised transport environment, not just the nominal design inputs. Together these features give both surrogates greater context to generalise across the parameter space.

2.6 Surrogate Modelling Methodology

2.6.1 Black-Box Baseline

The black-box surrogate predicts the flux curve $J(t)$ from simulation-derived inputs, with scalar transport quantities derived from the reconstructed curve. Principal component analysis (PCA) [22] is well suited to capture the key patterns in the flux curves without treating each time point independently, maintaining a compact low-dimensional representation that avoids overfitting. Ridge regression, a linear method with a penalty term, is then applied in this reduced space to predict the corresponding coefficients. Ridge regression is chosen over more complex alternatives such as random forests [23] or gradient boosting [24] given the limited training set of 700 samples. A regularised linear model generalises more reliably at this scale and remains interpretable through its retained coefficient structure [25], providing a clean baseline against which the physics-aware correction can be assessed. A neural network baseline was not used as the black-box, since a well-regularised linear model is a stronger and more interpretable baseline at this dataset size.

The workflow consists of four stages. First, the 20-dimensional input feature matrix (Table 2.2) is standardised. Second, flux curves are compressed using PCA, retaining 20 components beyond which additional components contribute less than 0.01% of variance. Third, a Ridge regression model is fitted in this reduced space with its penalty parameter selected by validation performance. Finally, the predicted representation is mapped back to flux curve space and scalar metrics derived from the reconstructed curve.

2.6.2 Physics-Corrected Hybrid Surrogate

Instead of replacing the black-box baseline with an independent model, the hybrid surrogate learns only the correction needed on top of it, adding physics-aware structure without discarding the baseline entirely [26]. The architecture is shown in Figure 2.4.

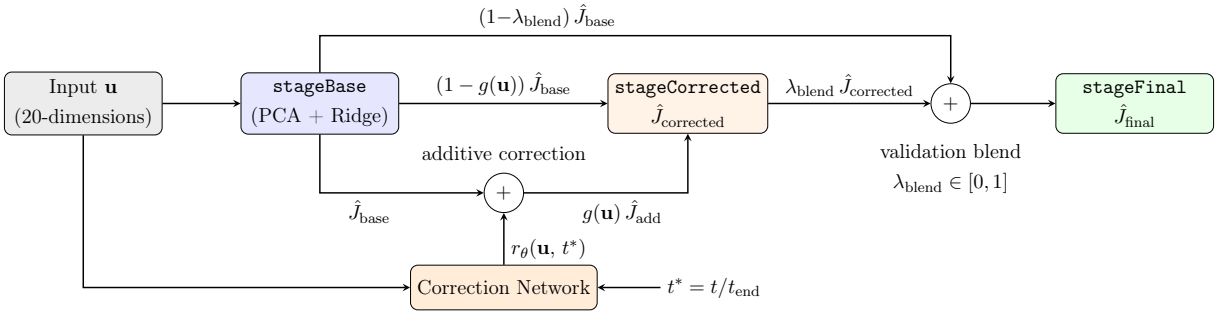


Figure 2.4: Hybrid surrogate architecture

`stageBase` is the black-box Ridge surrogate, producing $\hat{J}_{\text{base}}$ from the input parameters $\mathbf{u}$. The correction network r_θ is a feedforward neural network (layers of weighted connections passing signals in one direction) that learns how much to adjust $\hat{J}_{\text{base}}$ at each point in time, producing a bounded additive correction, $\hat{J}_{\text{add}}$, that cannot deviate vastly from the baseline:

$$\delta\hat{J}_\theta = s\,\tanh\big(r_\theta(\mathbf{u}, t^*)\big)\,\left|\hat{J}_{\text{base}}\right|, \qquad \hat{J}_{\text{add}} = \max\Big(0,\ \hat{J}_{\text{base}} + \delta\hat{J}_\theta\Big), \tag{2.9}$$

where $s > 0$ sets the maximum size of the correction and tanh smoothly maps the network output into the range $(-s,\, s)$. Scaling by $|\hat{J}_{\text{base}}|$ ensures the adjustment is always proportionate to the original shape of the baseline, so the correction refines rather than distorts it. The $\max(0, \cdot)$ clamp reflects the physical constraint that drug flux cannot be negative.

A learned gate $g(\mathbf{u}) \in [0, 1]$ (`stageCorrected`) then blends $\hat{J}_{\text{base}}$ and $\hat{J}_{\text{add}}$ on a per-input basis:

$$\hat{J}_{\text{corrected}} = (1 - g(\mathbf{u}))\,\hat{J}_{\text{base}} + g(\mathbf{u})\,\hat{J}_{\text{add}}. \tag{2.10}$$

Here, $g(\mathbf{u}) = 0$ returns the baseline unchanged, while $g(\mathbf{u}) = 1$ applies the full correction. `stageCorrected` therefore outputs the gated prediction $\hat{J}_{\text{corrected}}$, which is then combined with `stageBase` through a validation-selected blend weight λ_{blend} to produce `stageFinal`, allowing the final prediction to fall back toward the baseline if the corrected branch underperforms.

The training objective for r_θ and $g(\mathbf{u})$ combines five terms (full definitions in Appendix D):

$$\mathcal{L} = w_f\mathcal{L}_{\text{flux}} + w_a\mathcal{L}_{\text{anchor}} + w_n\mathcal{L}_{\text{nonneg}} + w_p\mathcal{L}_{\text{peak}} - w_g\bar{H}(g(\mathbf{u})) \tag{2.11}$$

where $\mathcal{L}_{\text{flux}}$ penalises deviation from the true flux, $\mathcal{L}_{\text{anchor}}$ penalises drift from the baseline prediction, $\mathcal{L}_{\text{peak}}$ penalises error in J_{peak} and t_{peak}, $\mathcal{L}_{\text{nonneg}}$ penalises negative flux values, and $\bar{H}(g)$ is the mean gate entropy, encouraging gate values near 0.5 early in training before decaying to allow later specialisation toward either the baseline or corrected branch.

2.6.3 Multi-Seed Training and Robustness

The hybrid surrogate is trained under three independent random seeds. Since the black-box baseline is deterministic, this allows the consistency of the hybrid's advantage to be assessed across repeated training runs, separating genuine improvement from stochastic variation.

2.7 Evaluation Metrics and Success Criteria

The evaluation combines three complementary layers to assess whether the surrogates approximate simulator outputs accurately, usefully, and in a physically reasonable way.

2.7.1 Curve-Level Metrics

The primary prediction target is the flux curve $J(t)$ at the bottom sink and is assessed by comparing predicted and true trajectories over time for each run. Four complementary metrics capture different aspects of predictive performance: relative L_2 error normalises prediction error by curve magnitude, accounting for differences in flux amplitudes across the parameter space. Root mean squared error (RMSE) measures absolute magnitude error at each time point. Mean absolute error (MAE) provides a complementary measure of error at individual time points. Pearson correlation measures temporal shape agreement, capturing whether the surrogate reproduces the dynamics of the flux curve independent of absolute offset. Relative L_2 error serves as the primary aggregate measure. Full discrete definitions are given in Appendix E.

2.7.2 Scalar Metrics

In addition to full-curve accuracy, the project evaluates three scalar transport quantities derived from the flux curve: peak flux J_{peak}, time to peak t_{peak}, and cumulative delivery $M_{\text{delivered},24h}$. These scalars capture pharmacokinetically relevant quantities that a model may fail to predict accurately even if overall curve reproduction is good. Three metrics assess performance: root mean squared error (RMSE) captures overall error magnitude; mean absolute error (MAE) reports the average absolute scalar error; and the coefficient of determination (R^2) indicates the proportion of variance explained. RMSE is used as the primary scalar error metric. Full definitions are given in Appendix E.

2.7.3 Physics Diagnostics

Beyond prediction accuracy, the evaluation of key diagnostics assesses whether surrogate outputs remain physically reasonable. Negative-flux violations check whether predicted flux ever becomes negative, which is physically inadmissible. Bottom-boundary flux consistency measures agreement between the surrogate-predicted flux curve and the simulator reference flux at the sink boundary. Mass-balance behaviour checks whether the rate of change of total drug mass is consistent with the net flux across the domain. Formal definitions are given in Appendix F. These are not primary performance metrics but serve as consistency checks, confirming that accuracy gains are not accompanied by physically implausible behaviour.

2.7.4 Runtime and Practical Utility

A key motivation for surrogate modelling is its computational acceleration benefits, so although prediction error is key, it is not the only aim to evaluate. Runtime is recorded for each component to enable an explicit comparison, covering simulation run time per execution and mean per-sample prediction time for each model stage. From this the speed-up ratio is calculated as simulator time divided by surrogate prediction time and reported alongside prediction-error metrics. The overhead ratio between the black-box and corrective surrogate inference times is also reported, reflecting the additional cost of the physics correction relative to the baseline. Although arguably a more accurate surrogate would be more worthwhile regardless of greater training cost, including the runtime in the evaluation reflects the practical trade-off directly rather than treating accuracy as the only measurement.

2.8 Reproducibility and Quality Assurance

Reproducibility. A major design factor of the project is reproducibility, as the core empirical claim depends on comparing the models under identical conditions. All simulation and training parameters are specified in YAML (a human-readable data format) configuration files to ensure every run can be re-executed from a single command with no implicit dependencies on environment state. The random seeds used for the multi-seed robustness evaluation are fixed and logged explicitly for dataset generation, train-test splitting, and all surrogate training runs. Deterministic splitting ensures that the black-box and hybrid surrogates share the same partitions for training and evaluation, allowing a proper performance comparison. Final trained model weights, fitted scalers, and diagnostic outputs are saved as versioned artefacts, allowing project outputs to be inspected or reproduced without rerunning the full pipeline.

Testing. Quality assurance is supported by unit tests covering the key components of the project: finite-difference operators, boundary-condition enforcement, numerical stability checks, transport-metric calculations, and dataset assembly routines. Each part is tested in isolation before it enters the pipeline, ensuring that errors in the simulator kernel or metric calculations do not propagate into the pipeline. Expected outputs and edge cases are both covered.

Version control. The codebase is managed using Git and hosted on GitHub. Development used a branch-per-feature workflow with `feature/`, `fix/`, `refactor/`, and `docs/` branch categories, with each phase of development corresponding to feature branches reviewed and merged after validation. Regular commits documented each change, making the commit history an auditable record of codebase evolution.

Project Structure and Development Approach. The project followed a milestone-based phased workflow reflecting the dependencies inherent in simulation-driven surrogate learning: simulator development and numerical verification (Phase 1), literature-anchored validation against published transdermal behaviour (Phase 2), extension to the heterogeneous finite-dose regime and dataset generation (Phase 3), and surrogate development and evaluation (Phase 4). This serial structure ensures that surrogate training data comes from a previously validated simulator, with errors identified before downstream use.

Chapter 3

Implementation and Validation

3.1 Software Architecture and Repository Organisation

The framework is organised as a modular, configuration-driven codebase that mirrors the staged pipeline introduced in Chapter 2: simulator design, dataset generation, surrogate training, and comparative evaluation. Each stage is implemented as a reusable module, allowing the full workflow to be executed consistently and inspected at every step.

The repository is divided into three layers: a core source package, a script layer, and YAML configuration files. The source package contains the simulator, numerical operators, boundary-condition handling, transport metrics, and interfaces for dataset handling and machine learning. The script layer exposes these as runnable pipeline stages, separating simulation, validation, dataset generation, and surrogate training into distinct entry points. Configuration files define the simulator regimes: specifying geometry, material properties, donor behaviour, and output controls. These saved configurations allow the `V1`, `V2`, and `V3` regimes to be reproduced seamlessly. The full repository layout is given in Appendix H.

The framework is implemented in Python 3.12. Core numerical operations use NumPy [27] and SciPy [28]. The black-box baseline uses Ridge regression with PCA via scikit-learn [29], and the physics-corrected surrogate is implemented in PyTorch [30]. The finite-difference solver is written from scratch with no external simulation library, giving full control over the discretisation and boundary-condition logic.

3.2 Simulator Implementation

3.2.1 Grid, Fields, and Boundary Handling

The simulator operates on the two-dimensional skin cross-section grid introduced in Chapter 2, with z the depth direction and x the lateral direction. The primary variable is the concentration field $C(z, x, t)$, stored alongside the effective diffusivity, clearance rate, and patch location fields. In the layered regimes the diffusivity varies by depth to represent the stratum corneum, viable epidermis, and dermis; the clearance field is only populated where dermal loss is active.

Boundary conditions are applied explicitly rather than incorporated into the numerical update. The top boundary is split by patch coverage: a Dirichlet (fixed-value) donor condition is imposed on-patch, and a zero-gradient condition is applied off-patch. The bottom boundary acts as a sink from which the flux $J(t)$ is recorded, and the lateral boundaries are no-flux, meaning no drug crosses the side walls. Keeping this logic separate from the main solver update ensures consistent enforcement across all three regimes.

3.2.2 Metrics and Run Artefacts

The main measured metric, bottom flux $J(t)$, is calculated numerically from the concentration gradient near the bottom boundary and averaged across the lateral direction. The other scalar quantities in Chapter 2 are then derived from the flux curve. Each run is saved as a bundle of `fields.npz`, `meta.json`, and `metrics.json`. This traceability allows downstream scripts to load, validate, or export data without having to rerun the solver.

3.2.3 Simulator Regimes

The three simulator regimes share a single solver codebase, with their differences specified through YAML configuration files. Table 3.1 gives the `V3` layer diffusivities, which differ from the `V2` calibrated values but are drawn from literature-informed estimates so that the dataset spans physiologically plausible parameter space rather than being anchored to a single calibrated point. In the layered regimes the dermis diffusivity is approximately 150 times that of the stratum corneum, which governs the explicit stability condition and requires a considerably finer timestep; `V1` avoids this by using a single uniform diffusivity throughout.

Table 3.1: Layer diffusivities for the `V3` regime (64×64 grid).

Layer	Grid rows	Thickness (μm)	D ($\mathrm{cm^2\,s^{-1}}$)
Stratum corneum	1	$\sim$8	1.3×10^{-9}
Viable epidermis	9	$\sim$72	1.0×10^{-8}
Dermis	54	$\sim$432	2.0×10^{-7}

3.3 Validation of the Simulator

The credibility of the machine-learning stages depends directly on the correctness of the simulator that generates their training data. Validation is therefore treated as a central part of the project rather than a secondary check, and is structured to mirror the progressive development of the simulator. The `V1` regime provides numerical verification in a simple homogeneous setting and comparison against an idealised analytic benchmark. The `V2` layered regime is calibrated against literature-reported macroscopic permeation behaviour, providing evidence of physiological plausibility. Together these give the justification for `V3`'s credibility as the basis for final dataset generation. `V3` extends this validated foundation with a finite-dose donor and variable patch geometry, producing flux curves with realistic depletion dynamics that the surrogate models are trained to capture.

3.3.1 `V1` Grid-Convergence Verification

The first validation step is on the homogeneous `V1` regime, which provides a simple setting for numerical verification. As the grid resolution is refined, the finite-difference scheme should produce solutions that converge toward a consistent result. A simulator that fails this cannot be trusted as a basis for surrogate training.

Grid-convergence verification was carried out by repeating the same baseline simulation on grids of $H \in \{16, 32, 64, 128, 256\}$ cells. At each refinement level, the L_2 difference between consecutive solutions was computed over time. The results are shown in Figure 3.1.

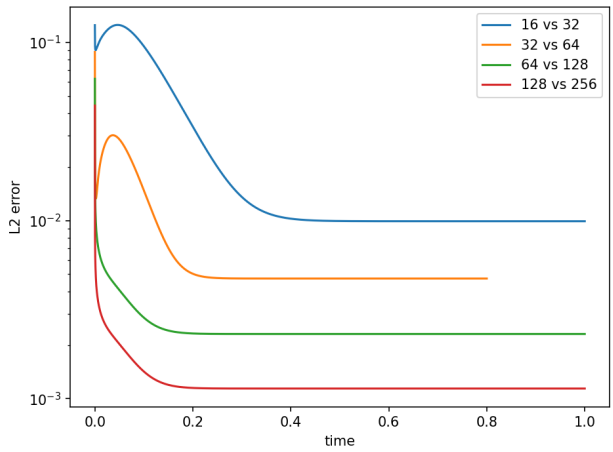


Table 3.2: Grid-convergence results.

Grid pair	Mean L_2	p[a]
16 vs 32	2.66×10^{-2}	—
32 vs 64	7.53×10^{-3}	1.82
64 vs 128	2.56×10^{-3}	1.56
128 vs 256	1.27×10^{-3}	1.01

[a] p is the observed convergence order between successive grid pairs.

Figure 3.1: Grid-convergence verification. L_2 differences between consecutive resolutions decrease with each refinement (*left*), with mean errors and convergence orders across grid pairs (*right*).

As observed, errors reduce at every grid refinement step, confirming the solver behaves consistently and that the 64×64 grid used in all simulations produces trustworthy results. However, the reduction is less than the ideal factor of four expected from a second-order scheme. As the grid is refined, the second-order bulk error decreases faster than the first-order error at the boundary cells; at fine resolutions the boundary error dominates, pulling the observed convergence order towards one. The final grid size was chosen as it converges at a practical computational cost, where further refinement would yield diminishing returns.

3.3.2 `V1` One-Dimensional Analytic Benchmark

The next `V1` validation step is a direct comparison against an idealised one-dimensional analytic benchmark derived from the Fourier-series solution of Crank [2] and adapted to the initial and boundary conditions (derived in Appendix G). Whereas grid convergence establishes internal consistency, this benchmark tests whether the solver reproduces physically expected diffusion behaviour. The depth-averaged concentration profile is compared against the analytic solution at 23 time points spanning early transient through near-steady-state.

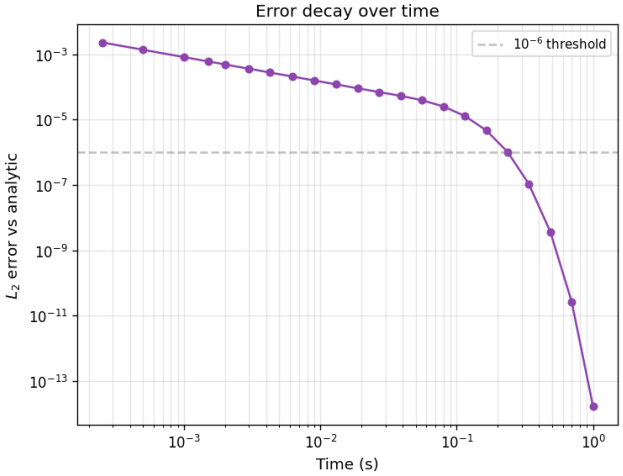


Table 3.3: 1D Benchmark results.

Metric	Value
Max L_2 error	2.316×10^{-3}
Mean L_2 error	3.201×10^{-4}
Final ($t = 1.0\,\mathrm{s}$)	1.664×10^{-14}

Figure 3.2: One-dimensional analytic benchmark for the `V1` regime. The L_2 error decays from the early peak to machine precision at steady state (*left*), with summary statistics (*right*).

The peak error occurs at early times when the concentration profile near the donor is at its steepest, before diffusion has smoothed it across the grid. From this peak the error drops steadily by eleven orders of magnitude, settling at machine precision at steady state. This confirms that the solver correctly tracks the diffusion dynamics throughout the full simulation, not just in the long-time limit, and provides confidence that the flux curves generated for surrogate training accurately reflect the underlying physics at every recorded time point.

3.3.3 `V2` Literature Calibration and Validation

Once the numerical baseline had been verified in `V1`, the next step was to assess whether the layered `V2` regime can be configured to reproduce macroscopic permeation behaviour reported in the literature. The calibration targets were taken from the lidocaine hydrochloride permeation data reported in Adamiak-Giera et al. [20]. Since layer-specific diffusivities are not present directly in the source paper and each diffusivity (D_{SC}, D_{VE}, D_{dermis}) simultaneously affects both permeability and lag time, inverse calibration using Nelder–Mead optimisation [31] was performed. The layers were tuned until the simulation permeability and lag time matched the literature targets. The results are given in Tables 3.4 and 3.5, with both targets reproduced to within 0.1% of their reported values, confirming near-exact agreement.

Table 3.4: `V2` Calibrated layer diffusivities

Layer	D ($\mathrm{cm^2\,s^{-1}}$)
Stratum corneum	5.55×10^{-10}
Viable epidermis	1.513×10^{-8}
Dermis	2.701×10^{-7}

Table 3.5: Calibration results against literature targets [20].

Quantity	Target	Simulated	Error
P ($\mathrm{cm\,s^{-1}}$)	7.380×10^{-7}	7.380×10^{-7}	-0.003%
T_{lag} (h)	1.401	1.402	$+0.056\%$

The calibrated diffusivities should be interpreted as effective parameters chosen to reproduce macroscopic transport within a physically plausible layered structure [9, 11] rather than unique anatomical values. The close agreement in Table 3.5 confirms the calibration was successful and the optimisation converged, rather than being an independent test of physical accuracy.

3.3.4 Justification of `V3` as the Final Simulator Regime

`V3` is built directly on the verified and calibrated predecessors `V1` and `V2`. It retains the layered structure established in `V2`, while using literature-informed diffusivities that preserve physiologically plausible ordering and magnitudes. It then extends this structure with a time-decaying donor, variable patch placement, spatial heterogeneity, and dermal clearance to create a richer machine-learning problem. The behavioural difference between regimes is illustrated in Figure 2.3. The final `V3` dataset remains grounded in this progressive validation chain, though the layer diffusivities are literature-informed rather than directly calibrated.

The baseline layer diffusivities in Table 3.1 differ from the `V2` calibrated values, the largest change is the stratum corneum diffusivity, which is 2.3× higher, but all remain within the natural inter-individual variation spanning roughly an order of magnitude [9, 1]. The physiological ordering $D_{\mathrm{SC}} < D_{\mathrm{VE}} < D_{\mathrm{dermis}}$ is preserved, and values sit within plausible ranges for skin transport [32].

3.4 Dataset Pipeline Implementation

Once the final regime had been established, the next stage was constructing a dataset pipeline for surrogate model training. The pipeline provides structured, consistently formatted inputs rather than raw simulation outputs. This preserves traceability between simulator configuration, run outputs and derived transport metrics so that the models can train effectively, whilst the export and assembly process remains reproducible.

3.4.1 Run Generation and Integrity Checking

The first part of the dataset pipeline is generating individual simulation runs from sampled parameter configurations under the `V3` regime. The final dataset contains 1,000 runs, partitioned deterministically into 700 training, 150 validation, and 150 test runs using a fixed-seed stratified split based on patch geometry labels. The split preserves an approximate balance across the discrete design variables, with scalar targets spanning several orders of magnitude. The QC plots confirming this are in Appendix I.

Figure 3.3 illustrates the diversity of flux curves present in the training set. The curves vary substantially in peak amplitude, rise time, and steady-state behaviour, driven primarily by patch geometry and finite-dose depletion rate. The variety in curve shape and magnitude is why the surrogate must capture both scalar summaries and full time-series outputs to represent the full range of behaviour.

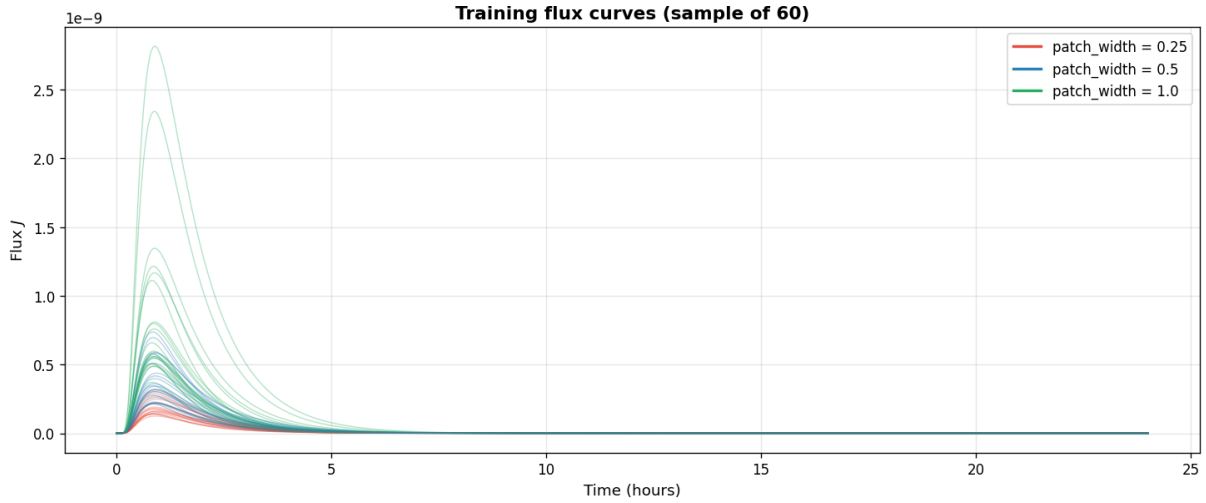


Figure 3.3: A random sample of 60 training flux curves, coloured by `patch_width` (red: 0.25, blue: 0.5, green: 1.0). The spread in amplitude, peak timing, and approach to steady state illustrates the behavioural diversity the surrogates must capture.

An explicit part of the pipeline is checking the integrity of the dataset: each run is first verified for the presence of required files (`fields.npz`, `meta.json`, `metrics.json`). Then metadata and key transport metrics are checked to ensure they can be parsed correctly and that field arrays are uncorrupted. A report identifying any failing runs is saved before assembling, which prevents faulty outputs from contaminating the datasets used for the actual machine learning later on. Detected failures are handled by dedicated integrity and repair scripts (`check_runs`, `fix_runs`) that identify and selectively regenerate problematic runs before assembly.

3.4.2 Assembly, Export and Quality Control

Once verified, the runs are assembled into a fixed processed dataset that sits between raw simulator outputs and the machine-learning training stage. Freezing this as a versioned intermediate ensures both surrogate models train and evaluate on exactly the same data. Without deterministic assembly, performance differences could reflect variation in the evaluation set rather than the models themselves. Fixed split fractions and a documented seed ensure the comparison is fair and reproducible across all reported experiments.

The final export stage produces the input feature matrix, full flux curve targets, and derived scalar transport quantities (J_{peak}, t_{peak}, $M_{\text{delivered,24h}}$) used for evaluation. Quality control is performed at this point to verify that exported arrays, feature definitions, target shapes, and split outputs remain consistent. This is crucial as otherwise errors introduced here could undermine both surrogate stages even if the underlying simulations were correct.

3.5 Surrogate Pipeline Implementation

The surrogate pipeline trains both models in sequence: the black-box baseline is trained first to establish a performance floor, and the physics-corrected hybrid is then built directly on top of it, taking the baseline prediction as its input rather than learning from scratch.

3.5.1 Black-Box Baseline Implementation

The black-box baseline predicts the flux curve $J(t)$ using PCA and Ridge regression. The model takes the 20-feature input matrix defined in Table 2.2, standardised using a StandardScaler fitted exclusively on the training split (shifting each feature to zero mean and scaling to unit variance), ensuring no information leaks into the normalisation and that no single feature dominates due to its raw magnitude.

Since flux magnitudes vary by several orders of magnitude across the dataset, a $\log_{10}$ transform is applied to the flux curve targets where the dynamic range exceeds a factor of 20. This prevents high-flux curves from disproportionately influencing the regressor at the expense of low-flux ones. PCA is then applied to reduce the dimensionality of the target space to 20 components, matching the input feature count whilst retaining sufficient variation for accurate flux curve reconstruction. A Ridge regressor is then trained in this reduced representation, with the Ridge penalty α selected by validation RMSE from a fixed candidate grid $\alpha \in \{0.001, 0.01, 0.1, 1, 10, 100\}$. In the saved final black-box run, the curve model selected a small penalty ($\alpha = 0.001$), indicating that only light regularisation was needed. The predicted PCA coordinates are then mapped back to the original output dimensionality to recover the full flux curve. The scalar transport quantities J_{peak}, t_{peak}, and $M_{\text{delivered,24}h}$ are then derived directly from the reconstructed flux curve rather than from separate regressors. This keeps scalar evaluation consistent with the primary prediction task.

Once trained, the baseline saves structured predictions for validation and test splits as `stageBase`, ensuring both models are evaluated against identical outputs. This ensures any improvement over `stageBase` is attributable to the corrective network alone.

3.5.2 Physics-Corrected Hybrid Implementation

The black-box baseline (`stageBase`) first maps the 20-feature vector from Table 2.2 to a baseline flux prediction $\hat{J}_{\text{base}}$. The corrected stage (`stageCorrected`) then takes $\hat{J}_{\text{base}}$, the same feature vector, and a normalised time coordinate $t^* = t/t_{\text{end}}$ as inputs to the neural network `RunConditionedCorrectiveSurrogate`, which produces a learned correction.

The network learns an additive correction to the baseline flux, parameterised using 48 Gaussian basis functions centred uniformly along the normalised time axis. This allows the correction to vary smoothly over time rather than independently at each step. The correction network contains four 128-unit Tanh layers, while the gate contains three 64-unit Tanh layers. Tanh activations, which output values in $(-1, 1)$, allow signed corrections in either direction while maintaining smooth outputs.

The correction is bounded to within a fraction s of the baseline magnitude through Equation 2.9, with s ramped from 0.3 to 0.5 during early training. In addition, the final output-layer weights are zero-initialised so that the network starts by producing no correction, anchoring it to the baseline before meaningful gradients develop. A learned gate $g(\mathbf{u}) \in [0, 1]$, defined in Equation 2.10, then blends the baseline and corrected predictions on a per-run basis according to where the correction is most reliable.

The weights are learned by minimising the objective in Equation 2.11 using the Adam optimiser [33] with a learning rate of 3×10^{-4}. The bounded correction, non-negativity clamp, and baseline anchoring provide the main structural physical constraints. Finally, `stageFinal` combines `stageBase` and `stageCorrected` using a blend weight λ selected on the validation set, which ranges from 0.9 to 1.0 across the reported seeds (7, 42, and 123). The full set of loss weights and schedule parameters used are given in Appendix D.

3.5.3 Diagnostics, Logging, and Output Artefacts

To support evaluation and comparison, both surrogates produce structured output artefacts. The predictions from the test set are stored in `pred_test.npz` with corresponding evaluation metrics in `metrics_test.json`. `summary.json` records the full run configuration alongside key summary metrics. Timing information is stored in `runtime.json`, and the hybrid model saves `history.json` for training convergence and `corrective_model.pt` for the trained network weights. This structure allows evaluation to operate directly on saved artefacts rather than rerunning the training pipeline, supporting traceability from reported results back to the corresponding predictions and configurations.

3.6 Testing and Reproducibility

The implementation has ten unit test modules, ensuring core components behave correctly. These cover boundary-condition handling, numerical operators, stability checks, transport-metric calculations, dataset assembly and export routines (see Appendix J). Reproducibility is enforced through YAML configuration files, recorded random seeds, and deterministic dataset splitting, ensuring simulator regimes and surrogate training runs can be reproduced exactly from saved configurations.

Chapter 4

Results, Evaluation, and Discussion

4.1 Evaluation Basis and Dataset Readiness

The final dataset for machine learning contains 1,000 simulation runs. These were split deterministically into 700 training, 150 validation, and 150 test runs. The dedicated validation split is necessary as the hybrid model includes model-selection decisions in the form of a blend-weight selection; this must be done independently of the test set to preserve the held-out test set as an unbiased basis for the final head-to-head comparison.

Quality-control checks confirm no split leakage, approximate balance of discrete design variables across splits, and finite target values for all primary scalar targets (see Appendix I).

The primary prediction target throughout is the full bottom-flux curve $J(t)$. The main scalar endpoints are J_{peak}, t_{peak}, and $M_{\text{delivered},24h}$, which capture the key characteristics of the finite-dose flux response (see Appendix K for graphical definitions).

4.2 Black-Box Baseline Results

The black-box model is a baseline reference against which the physics-corrected hybrid is evaluated. The strength of this baseline matters, as the hybrid's success should reflect a genuine gain over a competent surrogate, not simply an improvement over a less accurate one. Therefore, it needs to be a strong, interpretable model in its own right. As described previously, the black-box model uses `PCA + Ridge`. The full flux curve $J(t)$ is treated as the main prediction target, and the reported scalar metrics are derived from the reconstructed flux curve, consistent with the saved `metrics_test.json` artefacts.

For the primary $J(t)$ target, the baseline already performs strongly. On the held-out test set, it achieves a relative L_2 error of 0.0789, a curve-level root mean squared error of approximately 1.07×10^{-11}, and a Pearson correlation of 0.9967. These results indicate that the model captures the structure of the flux curves very well, from rise-to-peak shape down to finer transient structure with a high degree of accuracy. This therefore confirms that the final `V3` dataset carries enough structured variation that a data-driven surrogate can predict flux curves from the input features. Furthermore, it establishes that the hybrid comparison is not just a contest against a poor baseline. If the black-box model were performing badly from the outset, then later gains from a physics-corrected model would be much less convincing.

A similar pattern holds with the scalar quantities, with the baseline capturing all three targets well, as shown in Table 4.1. The accuracy of the scalars is more than just a secondary check; they define what is physically significant about the curve. The J_{peak}, t_{peak}, and $M_{\text{delivered},24h}$ accuracy means the model is not merely fitting the broad shape of the flux curve, but it is learning the specific characteristics that make that shape meaningful.

Curve metric	Value
Relative L_2 error	0.0789
RMSE	1.07×10^{-11}
MAE	2.90×10^{-12}
Pearson r	0.9967

Scalar target	MAE	RMSE	R^2
J_{peak}	3.85×10^{-11}	4.89×10^{-11}	0.9849
t_{peak}	31.2 s	48.50 s	0.8401
$M_{\text{delivered},24h}$	2.44×10^{-7}	3.20×10^{-7}	0.9828

Table 4.1: Black-box baseline performance on the held-out test set: curve-level metrics (left) and primary scalar targets (right).

The black-box baseline is genuinely competent in its own right. It predicts the primary curve target well and performs strongly on the most practically useful scalar transport quantities. Since the hybrid takes this baseline output as its starting point, any improvement it achieves represents a genuine corrective improvement - not a recovery from a weak foundation.

4.3 Physics-Corrected Hybrid Results

Rather than replacing the baseline, the hybrid applies a correction on top of it, with outputs organised into three stages: `stageBase` (baseline prediction), `stageCorrected` (physics-corrected prediction), and `stageFinal` (validation-selected blend). This structure lets the evaluation show not only whether the hybrid works but also how the improvement arises.

Using seed 42, which yields the strongest curve-level result among the three seeds evaluated, the results show a clear progression across stages. At `stageBase` the relative L_2 error is 0.0789, identical to the standalone baseline as expected. This falls to 0.0503 at `stageCorrected`, with `stageFinal` retaining this gain through the validation-selected blend ($\lambda = 1.0$), giving an overall reduction of approximately 36% over an already strong baseline. The Pearson correlation improves from 0.9967 to 0.9991, a meaningful gain given it occurs near the upper limit of the metric where further improvement is difficult.

The multistage pipeline supports the intended no-harm design. The final validation blend preserves the majority of the correction gain rather than collapsing back towards the baseline, confirming the corrective stage contributes real predictive gain rather than being undone by the blending step. The scalar results follow the same pattern. Improvement is present across all three primary targets, with the strongest gains on J_{peak} and $M_{\text{delivered},24h}$, the quantities most directly tied to the transient peak. The full results are given in Table 4.2.

Curve metric	Value
Relative L_2 error	0.0503
RMSE	6.84×10^{-12}
MAE	1.83×10^{-12}
Pearson r	0.9991

Scalar target	MAE	RMSE	R^2
J_{peak}	2.25×10^{-11}	3.08×10^{-11}	0.9940
t_{peak}	30.0 s	47.24 s	0.8483
$M_{\text{delivered},24h}$	1.54×10^{-7}	1.98×10^{-7}	0.9935

Table 4.2: Physics-corrected hybrid (`stageFinal`, seed 42) performance on the held-out test set: curve-level metrics (left) and primary scalar targets (right).

The corrective stage performs strongest on J_{peak} and $M_{\text{delivered},24h}$, consistent with the correction being concentrated in the transient and peak region. This reflects the hybrid's design - a targeted physical correction rather than a general improvement across the full curve. A full side-by-side comparison is given in the next section.

4.4 Comparative Evaluation of the Two Surrogate Strategies

Both models are evaluated individually on identical held-out test data. Therefore, the question at hand is not whether the models perform acceptably in their own right, but whether the physics-corrected hybrid delivers a meaningful improvement over a competent baseline.

Metric	Black-box	Hybrid (seed 42)	Improvement
Curve MAE	2.90×10^{-12}	$\mathbf{1.83 \times 10^{-12}}$	−37%
Curve RMSE	1.07×10^{-11}	$\mathbf{6.84 \times 10^{-12}}$	−36%
Curve relative L_2	0.0789	**0.0503**	−36%
Curve Pearson r	0.9967	**0.9991**	+0.0024
J_{peak} RMSE	4.89×10^{-11}	$\mathbf{3.08 \times 10^{-11}}$	−37%
t_{peak} RMSE	48.50 s	47.24 s	−3%
$M_{\text{delivered},24h}$ RMSE	3.20×10^{-7}	1.98×10^{-7}	−38%
J_{peak} MAE	3.85×10^{-11}	2.25×10^{-11}	−42%
t_{peak} MAE	31.2 s	30.0 s	−4%
$M_{\text{delivered},24h}$ MAE	2.44×10^{-7}	1.54×10^{-7}	−37%

Table 4.3: Head-to-head comparison of black-box and hybrid surrogate on the held-out test set. Bold indicates the better value.

The head-to-head table shows consistent gains across all primary metrics. The strongest improvements are on J_{peak} and $M_{\text{delivered},24h}$, with RMSE reductions of 37% and 38%. The gain on t_{peak} is more modest at 3%, for reasons discussed below. The consistency across MAE, RMSE, and relative L_2 confirms the improvement is not an artefact of any single metric. Figure 4.1 shows representative samples of model performance on the test set. The black-box predictions track the overall shape well but deviate visibly around the curve peak, whereas the hybrid shows consistently closer agreement with the simulator targets.

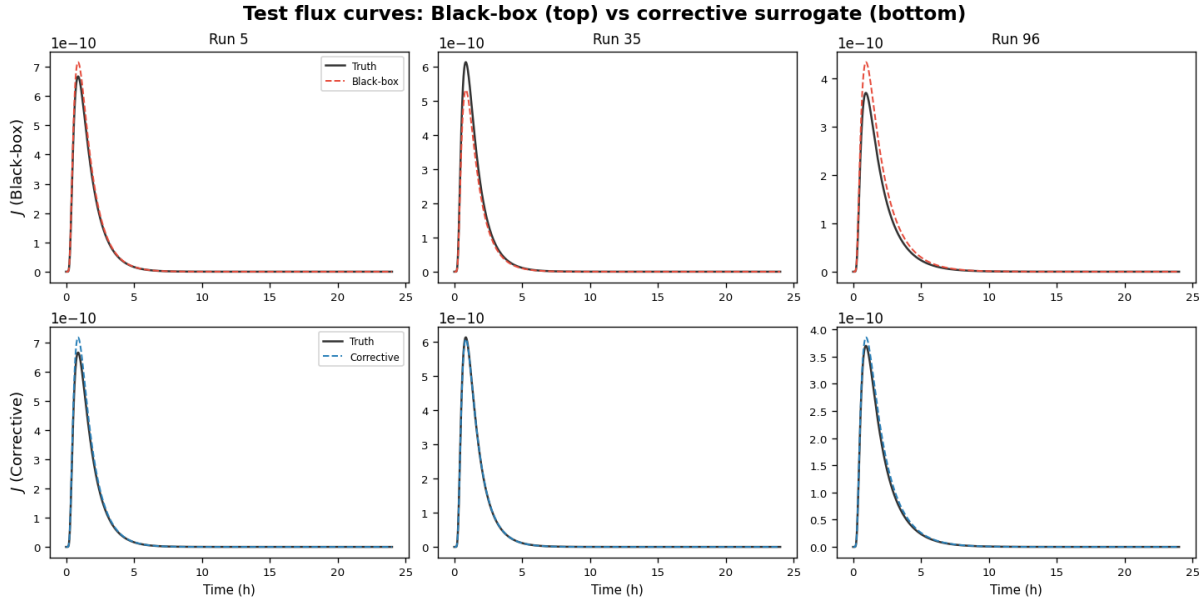


Figure 4.1: Representative held-out test flux curves for the black-box baseline (top row, red dashed) and hybrid final stage (bottom row, blue dashed) against simulator targets (solid black) across six test runs. Both models follow the curve well, but the hybrid improves on the baseline by producing visibly tighter agreement around the peak, where the baseline shows larger deviations.

Figure 4.2 shows where the hybrid's gains are most strongly concentrated. The largest reduction in mean absolute error occurs during the early transient and around the flux peak at approximately 1 h, which is precisely the region that drives J_{peak} and $M_{\text{delivered},24h}$. Beyond approximately 5 h both models converge to near-zero error as the flux decays, confirming the hybrid advantage is specific to the transient region rather than uniform across the full curve.

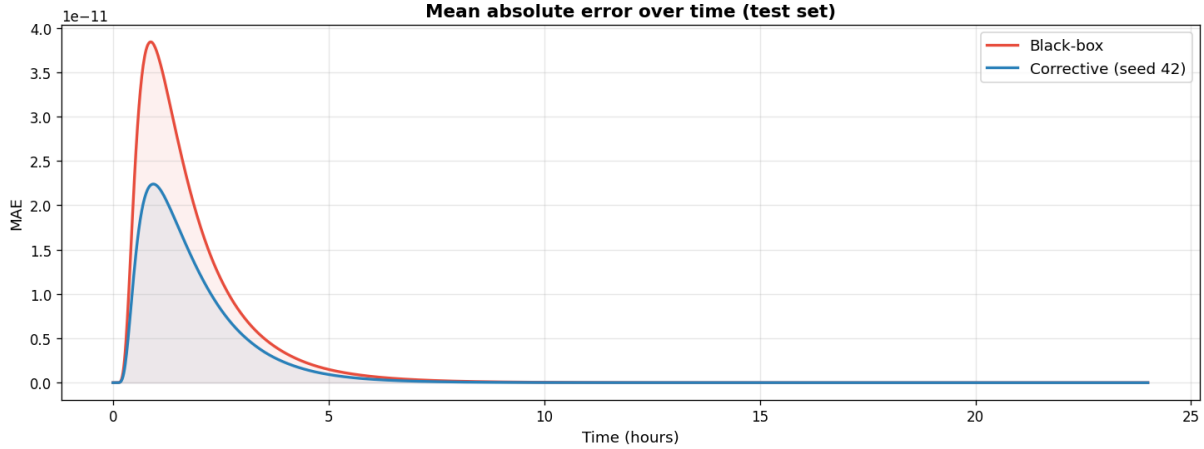
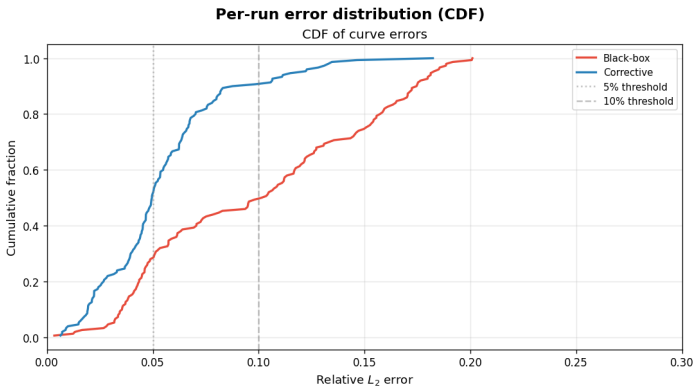


Figure 4.2: Mean absolute error over time across all 150 held-out test curves for the black-box baseline (red) and hybrid final stage (blue). Both models peak in error at approximately 1 h, coinciding with the flux peak, but the hybrid error is visibly lower throughout the transient region. Beyond approximately 5 h both curves converge to near-zero as the flux decays.

The empirical cumulative distribution function (CDF) of per-run relative L_2 error in Figure 4.3 shows the improvement is consistent across the full test set. The hybrid (blue) curve lies left of the black-box (red), indicating a greater proportion of runs achieve low error. The hybrid shifts the distribution strongly: 51.3% of predictions fall below the 5% threshold compared with 28.0% for the baseline, and 90.7% below 10% compared with 49.3%. The win/loss breakdown in the table within Figure 4.3 shows improvement in 82 of 150 runs (54.7%), with 50 showing regression (33.3%) and 18 near-unchanged ($|\Delta| \leq 0.005$), with the median per-run error falling from 0.102 to 0.050. The median improvement of 0.060 outweighs the median regression, confirming the gains are broadly distributed and not offset by the worsened minority. That over half of test runs improve and fewer than a third worsen, with a net mean Δ of -0.045, supports the conclusion that the hybrid delivers consistent benefit across the parameter space rather than selectively on easy cases.



Statistic	Value
Improved runs	82/150 (54.7%)
Worsened runs	50/150 (33.3%)
Near-unchanged	18/150 (12.0%)
Mean Δ	-0.045
Median Δ	-0.060
Hybrid below 5%	51.3%
Baseline below 5%	28.0%
Hybrid below 10%	90.7%
Baseline below 10%	49.3%

Figure 4.3: Empirical CDF of per-run relative L_2 error on the held-out test set (left) and per-run win/loss summary (right, $|\Delta| \leq 0.005$). The hybrid (blue) curve lies left of the black-box (red), indicating a greater proportion of runs achieve low error.

The scalar parity plots in Figure 4.4 show the hybrid producing visibly tighter scatter along the identity line for J_{peak} and $M_{\text{delivered},24h}$, corresponding to the 37% and 38% RMSE reductions reported in Table 4.3. For t_{peak}, the improvement is much smaller at 3%, with the parity plots having a discrete grid structure rather than a continuous scatter because peak time is constrained to the simulation time grid. This occurs as measurements around adjacent time steps can have near-identical flux values, so the `arg max` can shift by one grid step with only a very small change in the predicted curve. This means the t_{peak} pattern is defined in the simulated data, rather than indicating a major weakness specific to either model.

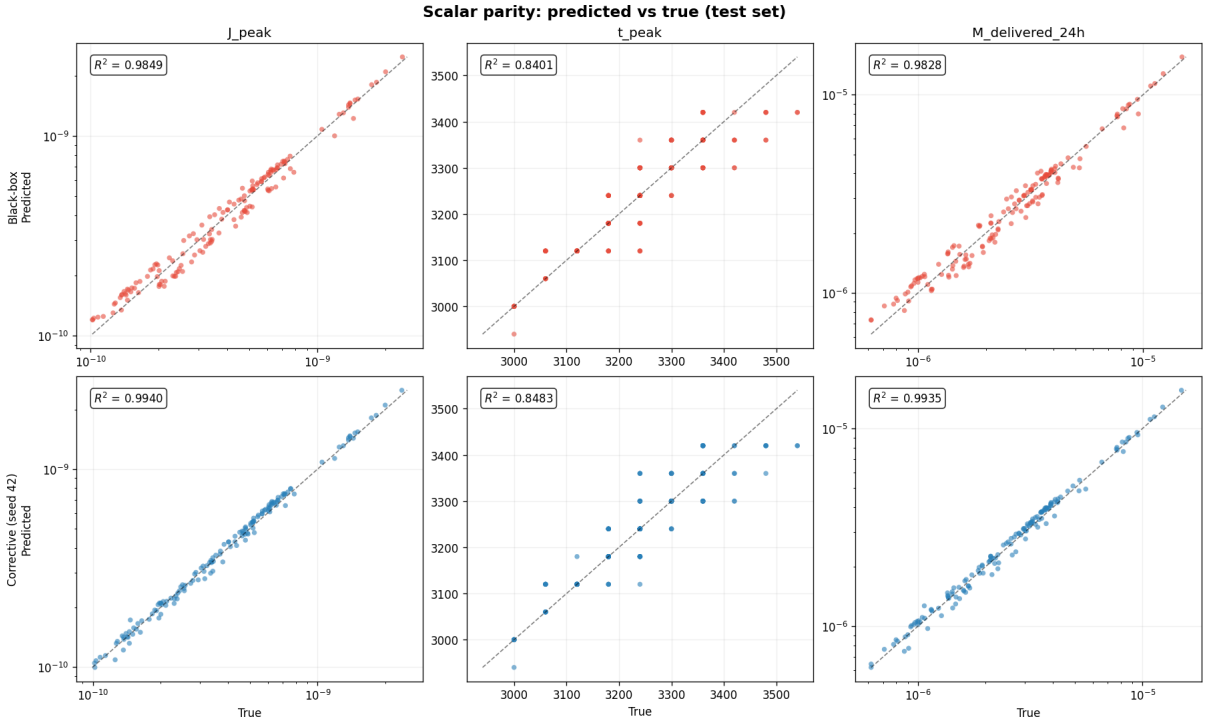


Figure 4.4: Parity plots (predicted vs true) for J_{peak}, t_{peak}, and $M_{\text{delivered},24h}$ on the held-out test set. Black-box baseline (top row, red) and hybrid final stage (bottom row, blue).

The physics diagnostics do not reveal obvious additional physical inconsistencies in the hybrid predictions (see Appendix F). Both models produce a negative-flux fraction of zero on the held-out test set. At the bottom-boundary level, the hybrid reduces the relative RMSE from approximately 0.0977 to 0.0533, a reduction of roughly 45%. The mass-balance relative RMSE of 1.221 is identical across the simulator reference and both surrogates, indicating that the metric variance is driven by the range of ground-truth values rather than surrogate error; the absolute mass-balance RMSE of 4.86×10^{-6} confirms that the corrective stage does not introduce detectable additional mass-conservation error. Together, these diagnostics indicate the hybrid improves boundary-level accuracy without introducing physical violations or degrading mass-conservation behaviour relative to the black-box baseline.

Overall the comparison supports a clear conclusion. The physics-corrected hybrid improves the primary curve target, the most practically informative scalar summaries, and the bottom-boundary physics diagnostics, while building on a baseline that was already strong. The gains are concentrated in the transient and peak-related quantities - consistent with both the physical character of the problem and the design intent of the corrective model.

4.5 Training Stability and Computational Cost

The comparative results so far show that the physics-corrected hybrid improves the black-box baseline on the held-out test set. A thorough evaluation must address two further questions: whether the observed improvement is stable across different random seeds, and what computational cost is associated with achieving that improvement when compared to the much simpler black-box baseline.

4.5.1 Multi-Seed Stability of the Hybrid Results

The hybrid was trained under seeds 42, 123, and 7. The results show the final curve-level performance is highly stable across these three runs with the final held-out test relative L_2 errors being 0.0503, 0.0507, and 0.0505 respectively. The mean final relative L_2 across seeds is approximately 0.0505. A coefficient of variation of 0.3% across the three independent runs supports the view that the curve-level gain is consistent across repeated hybrid training runs.

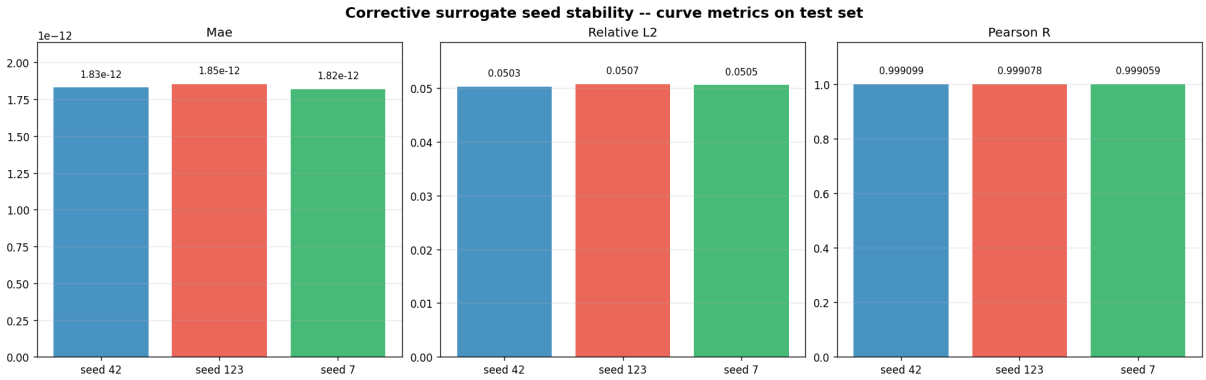


Figure 4.5: Hybrid model curve-level metrics across three random seeds. Near-identical bar heights confirm the improvement is not an artefact of a single favourable initialisation.

A similar pattern is observed for the scalar quantities. Their RMSE values across all three seeds are given in Table 4.4, with coefficients of variation remaining below 1% across all targets. These values indicate that the hybrid model's performance on the main transient and peak-related outputs remains stable under repeated training.

Scalar target	Seed 42	Seed 123	Seed 7	Coefficient of Variation
J_{peak} RMSE	3.08×10^{-11}	3.10×10^{-11}	3.08×10^{-11}	0.3%
t_{peak} RMSE	47.24 s	46.99 s	47.24 s	0.3%
$M_{\text{delivered},24h}$ RMSE	1.98×10^{-7}	2.00×10^{-7}	2.00×10^{-7}	0.5%

Table 4.4: Hybrid scalar target RMSE across three random seeds on the held-out test set.

Overall, the multi-seed analysis supports the conclusion that the hybrid model's advantage is reproducible rather than accidental. The method does not rely on a single favourable training trajectory to outperform the baseline on the main evaluation targets. The validation-selected blend weight was $\lambda = 1.0$ for seeds 42 and 123 and $\lambda = 0.9$ for seed 7, yet the resulting test performance is essentially indistinguishable, which suggests the corrected output is sufficiently reliable that the blending stage has little work to do. This consistency across seeds strengthens the validity of the seed 42 results used in the comparative evaluation, confirming they are representative rather than selectively favourable.

4.5.2 Computational Cost and Practical Trade-Offs

The second practical question concerns computational cost. One of the core motivations for surrogate modelling is to reduce the burden of repeated numerical simulation. Therefore it is not just enough to show the accuracy of the surrogates, but to compare the computation time related to each method and to consider what is acceptable within the scope of the project.

The saved timing artefacts record inference cost under identical evaluation conditions, with both surrogates evaluated on the same 150-run test split using 3 warm-up and 20 inference repeats.[1] The black-box achieves 0.011 ms per sample, giving a speedup of approximately 2,800,000× relative to the numerical simulator (≈29.41 s per run). The hybrid on CPU requires 2.42 ms per sample, a speedup of ∼12,200× over the simulator, at a 230× overhead relative to the black-box. On GPU the hybrid reduces to 0.071 ms per sample, narrowing the gap to ∼7× the black-box cost while achieving a ∼414,000× speedup over simulation. The black-box training step completes in 0.32 s and is negligible relative to either surrogate's deployment cost.

The practical interpretation of the inference cost difference depends on context. The black-box is faster at inference due to its simpler architecture, but both models are so far below the simulator cost that the difference is negligible for deployment. The 230× CPU overhead of the hybrid over the black-box narrows to ∼7× on GPU, meaning the additional inference cost of the physics correction becomes practically irrelevant in any hardware-accelerated setting.

For the scope of this project the inference overhead of the hybrid is acceptable. If minimising inference cost were the sole objective, the black-box would be preferred, but given the 36% improvement in curve accuracy and consistent gains across scalar transport metrics, the hybrid justifies its additional cost. At these inference speeds, parameter sweeps and optimisation loops that would be infeasible with the numerical simulator become computationally tractable. Training convergence plots for all three seeds are given in Appendix L.

4.6 Discussion in Relation to the Project Objectives

The results presented in this chapter address the research questions and objectives set out in Chapter 1. The simulator was progressively verified through numerical checks and analytic benchmarking, then anchored to literature through calibration, providing a credible basis for surrogate training. The dataset pipeline produced deterministic, leakage-free splits ensuring both models trained and evaluated under identical conditions. The black-box baseline met the requirement for a strong and interpretable reference model, and the physics-corrected hybrid delivered a measurable improvement on the primary curve target and the most informative scalar transport summaries. The multi-seed analysis confirmed the gains are reproducible, and the runtime analysis made the trade-off in accuracy and cost explicit rather than treating accuracy as the sole evaluation metric. The improvement is most pronounced on J_{peak} and $M_{\text{delivered},24h}$, the quantities most directly relevant to comparing patch delivery profiles, which aligns with the clinical motivation for accurate transient prediction established in Chapter 1.

[1] All timing measurements were recorded on a machine equipped with an Intel Core i7-12700 CPU and an NVIDIA GeForce RTX 4070 GPU (12 GB VRAM, CUDA 13.1).

There are important limits to these claims: the evaluation is confined to held-out test set performance using the processed `V3` dataset. Both models remain untested outside the sampled `V3` parameter space, so extrapolative performance is currently unknown. The runtime comparison is confined to inference cost; training cost is not directly compared as the two models differ substantially in architecture and optimisation procedure, meaning the full computational overhead of the hybrid exceeds the inference figures alone.

Overall, the evidence supports a measured but clear conclusion. The simulator framework provides a credible foundation through progressive verification (`V1`), literature calibration (`V2`), and extension to a heterogeneous regime (`V3`), with the dataset passing all quality-control checks prior to surrogate training. Within this setting, both surrogates demonstrate that the `V3` parameter space is learnable, but the central finding is that a physics-corrected hybrid can meaningfully improve on an already strong established baseline. The gains are concentrated in the transient and peak-related outputs, J_{peak}, t_{peak}, and $M_{\text{delivered},24h}$, which are the quantities most relevant to comparing delivery profiles in this modelling setting, though the improvement on t_{peak} is more modest as discussed above. These improvements are achieved with acceptable training stability and practical computational cost.

4.6.1 Limitations and Future Work

Although the present work demonstrates meaningful success within the scope of the project, several limitations and natural avenues remain.

One important limitation is the absence of experimental permeation data. The current performance does not establish how either surrogate would behave on real in-vitro or in-vivo measurements, where additional variability sources are present that the simulator does not capture. Extending the evaluation to such experimental datasets would strengthen the practical justification for the surrogate framework. Likewise, neither model has been tested beyond the sampled `V3` parameter space, so extrapolative behaviour remains uncertain.

On the methodological side, the hybrid enforces physics structurally rather than through active PDE-residual losses; a follow-on study comparing both configurations would isolate the contribution of each component. Neither model currently quantifies its own uncertainty, which is a meaningful limitation in clinical contexts where knowing how much to trust a prediction matters as much as the prediction itself. Additionally, the validation-selected blend did not substantially engage for two of three seeds ($\lambda = 1.0$), meaning the no-harm design did not actively limit the 33.3% regression rate observed on the test set; reducing per-run worsening remains an open design question.

Beyond the immediate scope of this project, the hybrid surrogate framework opens several promising directions. The most direct application would be coupling the surrogate to a dose optimisation loop, where the inference speedup makes it practical to sweep thousands of patch design configurations in the time a single simulation takes. Finally, the same surrogate architecture extends beyond transdermal delivery. The combination of a data-driven baseline with a structurally constrained corrective stage could transfer naturally to other PDE-governed pharmacokinetic settings. Other avenues include settings such as oral or intravenous delivery, where fast and physically plausible forward prediction is similarly valuable.

References

[1] Mark R. Prausnitz and Robert Langer. Transdermal drug delivery. *Nature Biotechnology*, 26(11):1261–1268, 2008. doi:10.1038/nbt.1504.

[2] John Crank. *The Mathematics of Diffusion*. Oxford University Press, Oxford, 2 edition, 1975.

[3] O.G. Jepps, Y. Dancik, Y.G. Anissimov, and M.S. Roberts. Modeling the human skin barrier — towards a better understanding of dermal absorption. *Advanced Drug Delivery Reviews*, 65(2):152–168, 2013. doi:10.1016/j.addr.2012.04.003.

[4] Russell O. Potts and Richard H. Guy. Predicting skin permeability. *Pharmaceutical Research*, 9(5):663–669, 1992. doi:10.1023/A:1015810312465.

[5] Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021. doi:10.1073/pnas.2101784118.

[6] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi:10.1038/s42254-021-00314-5.

[7] Xuhui Meng and George Em Karniadakis. A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse PDE problems. *Journal of Computational Physics*, 401:109020, 2020. doi:10.1016/j.jcp.2019.109020.

[8] Raphaël Pestourie, Youssef Mroueh, Chris Rackauckas, Payel Das, and Steven G. Johnson. Physics-enhanced deep surrogates for partial differential equations. *Nature Machine Intelligence*, 5(12):1458–1465, 2023. doi:10.1038/s42256-023-00761-y.

[9] Samir Mitragotri, Yuri G. Anissimov, Annette L. Bunge, H. Frederick Frasch, Richard H. Guy, Jonathan Hadgraft, Gerald B. Kasting, Majella E. Lane, and Michael S. Roberts. Mathematical models of skin permeability: an overview. *International Journal of Pharmaceutics*, 418(1):115–129, 2011. doi:10.1016/j.ijpharm.2011.02.023.

[10] Robert J. Scheuplein. Mechanism of percutaneous absorption. II. Transient diffusion and the relative importance of various routes of skin penetration. *Journal of Investigative Dermatology*, 48(1):79–88, 1967. doi:10.1038/jid.1967.11.

[11] Yuri G. Anissimov, Owen G. Jepps, Yuri Dancik, and Michael S. Roberts. Mathematical and pharmacokinetic modelling of epidermal and dermal transport processes. *Advanced Drug Delivery Reviews*, 65(2):169–190, 2013. doi:10.1016/j.addr.2012.04.009.

[12] K. W. Morton and D. F. Mayers. *Numerical Solution of Partial Differential Equations: An Introduction*. Cambridge University Press, Cambridge, 2 edition, 2005.

[13] A.I.J. Forrester and A.J. Keane. Recent advances in surrogate-based optimization. *Progress in Aerospace Sciences*, 45(1):50–79, 2009. doi:10.1016/j.paerosci.2008.11.001.

[14] Haitao Liu, Yew-Soon Ong, Xiaobo Shen, and Jianfei Cai. When Gaussian process meets big data: A review of scalable GPs. *IEEE Transactions on Neural Networks and Learning Systems*, 31(11):4405–4423, 2020. doi:10.1109/TNNLS.2019.2957109.

[15] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378(1):686–707, 2019. doi:10.1016/j.jcp.2018.10.045.

[16] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi:10.1016/j.jcp.2021.110768.

[17] Christopher Rackauckas, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, Ali Ramadhan, and Alan Edelman. Universal differential equations for scientific machine learning. arXiv preprint arXiv:2001.04385, 2021. Available at: https://arxiv.org/abs/2001.04385.

[18] Marc C. Kennedy and Anthony O'Hagan. Bayesian calibration of computer models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 63(3):425–464, 2001. doi:10.1111/1467-9868.00294.

[19] Yuri G. Anissimov and Michael S. Roberts. Diffusion modeling of percutaneous absorption kinetics: 2. finite vehicle volume and solvent deposited solids. *Journal of Pharmaceutical Sciences*, 90(4):504–520, 2001. doi:10.1002/1520-6017(200104)90:4<504::AID-JPS1008>3.0.CO;2-H.

[20] Urszula Adamiak-Giera, Anna Nowak, Wiktoria Duchnik, Paula Ossowicz-Rupniewska, Anna Czerkawska, Anna Machoy-Mokrzyńska, Tadeusz Sulikowski, Łukasz Kucharski, Marta Białecka, Adam Klimowicz, and Monika Białecka. Evaluation of the in vitro permeation parameters of topical ketoprofen and lidocaine hydrochloride from transdermal Pentravan® products through human skin. *Frontiers in Pharmacology*, 14:1157977, 2023. doi:10.3389/fphar.2023.1157977.

[21] Suhas V. Patankar. *Numerical Heat Transfer and Fluid Flow*. McGraw-Hill, New York, 1980.

[22] Ian T. Jolliffe. *Principal Component Analysis*. Springer, New York, 2 edition, 2002.

[23] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi:10.1023/A:1010933404324.

[24] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001. doi:10.1214/aos/1013203451.

[25] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi:10.1080/00401706.1970.10488634.

[26] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, Las Vegas, NV, USA, 2016. IEEE. doi:10.1109/CVPR.2016.90.

[27] Charles R Harris, K Jarrod Millman, Stefan J van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi:10.1038/s41586-020-2649-2.

[28] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi:10.1038/s41592-019-0686-2.

[29] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL: http://jmlr.org/papers/v12/pedregosa11a.html.

[30] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. Article no. 721. URL: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

[31] J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965. doi:10.1093/comjnl/7.4.308.

[32] Charlotte A Ellison et al. Partition coefficient and diffusion coefficient determinations of 50 compounds in human intact skin, isolated skin layers and isolated stratum corneum lipids. *Toxicology in Vitro*, 69:104990, 2020. doi:10.1016/j.tiv.2020.104990.

[33] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL: https://arxiv.org/abs/1412.6980.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

This project set out to build and validate a transdermal diffusion simulator, then test whether a physics-corrected hybrid surrogate could outperform a strong black-box baseline. All four principal objectives were met, though a number of limitations are worth acknowledging.

Simulator. The calibration of `V2` to within 0.1% of published literature targets was the aspect I was most pleased with and represents a stronger anchor than many surrogate-training studies provide. The simulator remains an effective-parameter representation rather than an anatomically realistic one, extending to `V3` introduces further approximations. Purely passive diffusion is assumed and the donor is approximated by an exponential decay rather than a physically derived depletion model. These were appropriate for a controlled surrogate comparison and the scope I had but limited real-world applicability, and if I were approaching this again, I would explore a more physically grounded donor model from the outset.

Dataset and sampling. By generating 1,000 runs I balanced dataset size with generation time, but in hindsight it is small with 700 training samples for a neural network correction stage. The discrete geometric configurations are the more fundamental limitation; with only three patch widths and three offset positions the surrogate is essentially interpolating between a small number of fixed geometries rather than learning a truly continuous geometric response. Next time I would prioritise a larger and more geometrically varied dataset. I would also run a data ablation study by training on progressively smaller subsets to understand the minimum viable dataset size. From this I would test explicitly on out-of-distribution configurations to understand where generalisation breaks down. Together these improvements would tell us how robust the framework actually is beyond the sampled `V3` parameter space.

Surrogate evaluation. Three seeds provides meaningful stability evidence but falls short of a full robustness study. The weak t_{peak} improvement was something I initially attributed to the model, but I now understand it reflects the simulation time grid structure rather than a surrogate weakness. The physics component enforces constraints structurally rather than through active PDE residuals; a true PINN formulation would offer stronger physical grounding at the cost of greater training complexity. Finally, the validation blend selected $\lambda = 1.0$ for two of three seeds, meaning it effectively did not engage in those runs. I would argue this reflects sound defensive design, but it raises a legitimate question about whether the blending stage is necessary in this setting and would be worth quantitatively testing.

Software and testing. Targeted unit tests covered each independent step of the framework. The decision not to build a full end-to-end integration test was a conscious trade-off prioritising development speed over automated pipeline coverage. Looking back, the staged approach worked but introduced more overhead than anticipated as the pipeline grew. I would now treat an integration test as a first-class deliverable rather than an afterthought, writing it in parallel with the pipeline from day one.

A.2 Personal reflection and lessons learned

The decision to build a modular, configuration-driven codebase from the outset proved more valuable than I expected. When extending to new simulator regimes and surrogate stages, the core package required minimal changes - something that would not have been possible with a less structured approach. This taught me that upfront structural decisions have a compounding effect and that the time invested early saved significantly more later.

Early in the project I explored a physics-informed (PINN) style formulation, embedding PDE residuals directly into the training objective. In practice the loss-weighting sensitivity made stable training difficult - small changes to the relative weight of the physics and data terms produced very different behaviours, consistent with the instability issues documented in the literature. This experience directly informed the decision to pursue a structurally constrained hybrid instead, where the physics are enforced through the architecture rather than through competing loss terms.

The validation blend result, where λ is selected on the validation set to determine how much of the correction to retain, also surprised me. I expected λ to actively moderate the correction on runs where it was less reliable, but it selected the full correction almost universally across seeds. This makes me question whether the gating mechanism added meaningful value in this setting. A simpler correction architecture without the learned gate might perform similarly, and testing this directly would be a worthwhile follow-on study. It also raises the broader question of whether the correction stage could function as a standalone model rather than being built on the baseline, though the anchoring behaviour during training suggests the baseline dependency plays an important stabilising role.

The validation stage was the part of the project I found most rewarding. Building a progressive chain of numerical verification, analytic comparison, and literature calibration gave the surrogate results a credibility that a single training run on an unvalidated dataset would not have had. I came to appreciate that in simulator-driven machine learning, the quality of the data pipeline matters as much as the quality of the model. This understanding has changed how I think about applied machine learning more broadly.

I also learned to be more precise about what evaluation metrics can and cannot establish. Early in the surrogate evaluation, P and J_{ss} were included as scalar targets alongside J_{peak}, t_{peak}, and $M_{\text{delivered},24h}$. Their R^2 values proved unstable across seeds, which was initially attributed to training variability. Examining the physical meaning revealed the actual cause: both quantities presuppose a steady-state flux plateau that a finite-dose regime never reaches, making the targets themselves ill-defined for `V3` runs. This led to the decision to remove P and J_{ss} from the `V3` scalar evaluation entirely and restrict scalar metrics to quantities that are well-defined under finite-dose conditions. Recognising that apparent model instability can reflect a physical mismatch rather than a training problem, and acting on that by revising the evaluation scope, was a useful lesson in applying domain knowledge to guide methodological decisions.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

All code produced in this project is original work written without the use of external code, scripts, or components from third-party repositories. External libraries such as NumPy, SciPy, PyTorch, and scikit-learn are used and are distributed under permissive open-source licences (BSD, MIT, or Apache 2.0) that permit academic use without restriction. No proprietary software or unlicensed datasets were used. The literature permeability and lag-time values used as calibration targets in the `V2` regime are taken from a published open-access journal article [20] and are used solely as numerical benchmarks. No personal data of any kind was collected or processed, so data-protection legislation (such as GDPR) is not applicable to this project. All published equations, values, and results cited throughout the report are used for academic purposes and fall within fair dealing provisions for non-commercial research.

The framework developed here is purely research software with no clinical deployment intended. However, were a surrogate model of this kind ever used to inform real patch design or dosing decisions, it would likely fall under medical device software regulations. This includes medical device software regulations such as the EU Medical Device Regulation (MDR) which require formal safety validation, risk management, and quality assurance processes that fall well beyond the scope of academic research software. The current work makes no claims of clinical applicability, and any such use would require substantially more extensive regulatory validation before deployment.

A.3.2 Social issues

Transdermal drug delivery is used in a wide range of clinical contexts including pain management, nicotine replacement, and hormone therapy. Computational tools that can predict drug flux curves and peak delivery quantities more efficiently could, in principle, support patch design and reduce the need for repeated experimental testing, potentially contributing to the development of safer and more effective transdermal formulations. A simulation-driven approach also carries an indirect environmental benefit by reducing reliance on physical experimental iterations, giving a smaller material and energy footprint than repeated laboratory testing. Patients who depend on transdermal delivery for chronic conditions stand to benefit most directly from more efficient patch design tools. The open-source foundation of this framework also means the approach is reproducible by other researchers without requiring expensive proprietary software.

However, the simulator and surrogate models presented here operate entirely on synthetic data within a controlled computational regime, and the framework has not been validated against clinical populations or experimental measurements beyond a single literature calibration scenario. This synthetic scope limits both the applicability and any potential for misuse. The outputs are not directly transferable to real drug delivery decisions without substantially more extensive validation, and no negative social consequences are anticipated from the work in its current form.

A.3.3 Ethical issues

This project involves no human participants, no animal studies, and no collection of personal or sensitive data. All simulation data are generated synthetically from the implemented computational model. The literature data used for calibration are from a publicly available peer-reviewed study and are used only as numerical targets, not as patient data. Because all data are synthetically generated and no human or animal subjects are involved, there are no ethical concerns relating to participant consent, data privacy, or research involving living subjects, and standard ethical approval requirements do not apply to this project.

The project aims to be transparent about the limitations of its findings. The results are presented with explicit acknowledgement of the boundaries of what can be claimed. For example, that performance is demonstrated using data only generated from the `V3` simulator regime, and that generalisation to experimental data remains unvalidated. This is consistent with standard responsible reporting practice. Due to the project's clinical relevance, there is an ethical obligation to be clear about what the model can and cannot establish. Presenting results with the scope explicitly listing limitations is far better than just implying broader applicability than the evidence supports. This practice was adopted throughout the project.

A.3.4 Professional issues

The project was conducted in accordance with the professional standards expected at the university. Code was developed using version control with a structured branching workflow, separating features, fixes, refactors, and documentation. All experiments were made reproducible through configuration files, recorded random seeds, and deterministic dataset splitting. Results were reported honestly, including negative or inconclusive findings (such as the instability of P and J_{ss} across seeds and the modest gain in t_{peak}), rather than selectively presenting only favourable outcomes. This extends to reporting that 33.3% of test runs worsened under the hybrid and that the validation blend selected $\lambda = 1.0$ for two of three seeds, rather than redesigning the evaluation to conceal these results.

These practices are consistent with the BCS Code of Conduct, which emphasises public interest, professional competence, honesty, and integrity. In particular, the explicit acknowledgement of software testing limitations (no full end-to-end integration test) and the careful distinction between what the calibration establishes and what it does not reflect the professional obligation to avoid overstating the reliability or scope of technical claims. In a project touching on drug delivery, the public interest dimension of the BCS Code is particularly relevant. The explicit scope limitations presented throughout this work reflect a responsibility not to overstate findings in a domain where misleading claims could have real consequences.

Appendix B

External Material

All code produced in this project was written entirely by the author. No external code, scripts, or software components from outside sources were used in the final project.

Standard open-source Python libraries were used throughout, including NumPy, SciPy, PyTorch, and scikit-learn. These are used as third-party dependencies in the conventional software-engineering sense and do not constitute external materials specific to this project.

The only external data used are the published permeability and lag-time values for lidocaine hydrochloride reported in Adamiak-Giera et al. [20], which serve as numerical calibration targets for the `V2` simulator regime. These values are taken from an open-access journal article and are used solely as benchmark quantities; no dataset or raw experimental data was obtained from the authors. Published literature was consulted throughout for equations, parameter values, and benchmark results, as cited in the main text.

Appendix C

Discretisation of the Bottom Flux

The simulator is set up to operate on a two-dimensional grid. It is defined as $H \times W$ cells, indexed by depth row $i \in \{1, \ldots, H\}$ and lateral column $j \in \{1, \ldots, W\}$, with uniform spacing Δz in the depth direction. This grid represents a vertical cross-section of the skin, where rows correspond to increasing depth from the surface and columns capture lateral variation.

The bottom boundary at row H enforces a sink condition $C_{H,j}^{(n)} = 0$ at all time steps n. This means that any drug concentration reaching the deepest layer is immediately removed, representing the assumption that drug absorbed at the skin–bloodstream interface is instantly carried away by systemic circulation and does not accumulate.

Step 1: Continuous flux. The flux leaving the domain through the lower boundary follows from Fick's first law [2]:

$$J(t) = -D \left.\frac{\partial C}{\partial z}\right|_{z=L_z}.$$

Step 2: Finite-difference approximation. The concentration gradient at the lower boundary is estimated by taking the difference in concentration between the last interior row $H-1$ and the sink row H, divided by the grid spacing Δz:

$$\left.\frac{\partial C}{\partial z}\right|_{z=L_z} \approx \frac{C_{H,j}^{(n)} - C_{H-1,j}^{(n)}}{\Delta z}.$$

Step 3: Interface diffusivity. In the layered `V2` and heterogeneous `V3` regimes, diffusivity varies between cells. Therefore, the flux is computed at the boundary between rows $H-1$ and H, so the local diffusivity $D_{H-1,j}$ from that last interior row is used.

Step 4: Lateral averaging. Each lateral column produces its own flux estimate, but a single representative value is needed for the whole boundary. Taking the mean across all W columns gives the final discrete approximation used throughout the simulator:

$$J(t_n) \approx -\frac{1}{W} \sum_{j=1}^{W} D_{H-1,j} \frac{C_{H,j}^{(n)} - C_{H-1,j}^{(n)}}{\Delta z}.$$

Appendix D

Training Objective: Component Loss Definitions

The full training objective is epoch-dependent:

$$\mathcal{L}(e) = w_f\, c(e)\, \mathcal{L}_{\text{flux}} + w_a\, c(e)\, d_a(e)\, \mathcal{L}_{\text{anchor}} + w_n\, \mathcal{L}_{\text{nonneg}} + w_p\, c(e)\, \mathcal{L}_{\text{peak}} - w_g\, d_g(e)\, \bar{H}(g). \quad \text{(D.1)}$$

where e denotes epoch, $c(e)$ is a curriculum warmup factor that ramps from 0 to 1 over the first three epochs, $d_a(e)$ is an exponential decay applied to the anchor term, and $d_g(e)$ is an exponential decay applied to the gate regulariser. The terms address different aspects of the training signal: flux accuracy, baseline anchoring, physical non-negativity, peak supervision, and early gate entropy maximisation respectively.

Component Loss Definitions

Flux loss $\mathcal{L}_{\text{flux}}$ - primary reconstruction target. Penalises disagreement between predicted and true flux curves in the asinh-transformed domain, which compresses the dynamic range across runs with very different flux magnitudes:

$$\mathcal{L}_{\text{flux}} = \frac{1}{N_f} \sum_{i=1}^{N_f} \Big[\operatorname{asinh}(\alpha \hat{J}_i) - \operatorname{asinh}(\alpha J_i)\Big]^2.$$

Anchor loss $\mathcal{L}_{\text{anchor}}$ - keeps the correction close to zero early in training. Penalises large deviations of the corrected prediction from the black-box baseline, decaying exponentially so that the network is progressively freed to learn meaningful corrections as training proceeds:

$$\mathcal{L}_{\text{anchor}} = \frac{1}{N_f} \sum_{i=1}^{N_f} \Big[\operatorname{asinh}(\alpha \hat{J}_i) - \operatorname{asinh}(\alpha \hat{J}_{\text{base},i})\Big]^2.$$

Non-negativity loss $\mathcal{L}_{\text{nonneg}}$ - enforces the physical constraint that flux cannot be negative. Penalises any predicted flux value below zero:

$$\mathcal{L}_{\text{nonneg}} = \frac{1}{N_f} \sum_{i=1}^{N_f} \operatorname{ReLU}(-\hat{J}_i)^2.$$

Peak loss $\mathcal{L}_{\text{peak}}$ - provides direct supervision on the two scalar transport quantities most relevant to comparing delivery profiles. Penalises relative error in predicted peak flux and peak timing:

$$\mathcal{L}_{\text{peak}} = \frac{1}{N_b} \sum_{j=1}^{N_b} \left(\frac{\hat{J}_{\text{peak},j} - J_{\text{peak},j}}{|J_{\text{peak},j}| + \varepsilon} \right)^2 + \frac{1}{N_b} \sum_{j=1}^{N_b} \left(\frac{\hat{t}_{\text{peak},j} - t_{\text{peak},j}}{|t_{\text{peak},j}| + \varepsilon} \right)^2.$$

Peak quantities are extracted using a soft peak operator.

Gate entropy regularisation - regularises the learned gate $g(\mathbf{u}) \in [0, 1]$ by maximising entropy early in training. This encourages gate values near 0.5, so that the model initially balances the baseline and corrective branches rather than committing prematurely to either one. The entropy term is then decayed over training, allowing the gate to specialise where the correction is useful:

$$\bar{H}(g) = -\frac{1}{N_b} \sum_{j=1}^{N_b} \left[g_j \log g_j + (1 - g_j) \log(1 - g_j) \right], \qquad g_j = g(\mathbf{u}_j).$$

The objective includes this with a negative sign, $-w_g d_g(e) \bar{H}(g)$, so that entropy is maximised rather than minimised.

Here $\hat{J}_{\text{base}}$ is the black-box backbone prediction, $\hat{J}$ is the corrective-stage flux prediction, α is the asinh flux scaling constant, N_f is the number of sampled flux points per batch, and N_b is the number of runs used for peak and gate supervision.

Final Hyperparameter Values

The weight values and schedule parameters used across all three reported seeds (42, 123, and 7) are identical and are given in Table D.1.

Parameter	**Value**	**Notes**
w_f	10.0	Flux reconstruction weight
w_a	0.05	Anchor weight
w_n	0.02	Non-negativity weight
w_p	3.0	Peak supervision weight
w_g	0.05	Gate entropy-maximisation weight
α	10^{11}	asinh flux scaling constant
Epochs	800	Total training epochs
s_{init}	0.3	Initial correction bound
s_{final}	0.5	Final correction bound
Bound ramp	epochs 1–400	Linear ramp $0.3 \to 0.5$, fixed at 0.5 thereafter
Curriculum $c(e)$	warmup = 1, ramp = 2	Ramps $0 \to 1$ over epochs 1–3, fixed at 1 thereafter
Anchor decay $d_a(e)$	$\exp\left(-\frac{0.693\,e}{133.3}\right)$	Half-life at epoch ≈133 (epochs/6)
Gate decay $d_g(e)$	$\exp\left(-\frac{0.693\,e}{240}\right)$	Half-life at epoch 240 (epochs × 0.3)

Table D.1: Training objective weight values and schedule parameters for the final reported hybrid surrogate runs. Values are identical across seeds 42, 123, and 7.

The weight magnitudes reflect the intended priority ordering: flux reconstruction carries the largest weight ($w_f = 10.0$), peak supervision the second largest ($w_p = 3.0$), and the anchor, non-negativity, and gate terms carry small weights since their role is structural regularisation rather than driving the primary prediction task. The schedule parameters are designed to give the network a stable starting point, with the curriculum, anchor decay, and correction-bound ramp progressively relaxing constraints as training stabilises.

Appendix E

Evaluation Metric Definitions

Curve-Level Metrics

Let $J_{i,n}$ denote the reference flux and $\hat{J}_{i,n}$ the predicted flux for run i at time step n. The dataset contains N runs each with T time points.

$$\mathrm{MAE} = \frac{1}{NT} \sum_{i=1}^{N} \sum_{n=1}^{T} \left| \hat{J}_{i,n} - J_{i,n} \right|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{NT} \sum_{i=1}^{N} \sum_{n=1}^{T} \left(\hat{J}_{i,n} - J_{i,n} \right)^2}$$

$$\mathrm{rel}L_2 = \frac{\left(\sum_{i,n} \left(\hat{J}_{i,n} - J_{i,n} \right)^2 \right)^{1/2}}{\left(\sum_{i,n} J_{i,n}^2 \right)^{1/2} + \varepsilon}$$

where $\varepsilon = 10^{-12}$ is a small constant included for numerical stability.

$$r = \frac{\mathrm{cov}\left(\mathrm{vec}(\hat{J}), \mathrm{vec}(J) \right)}{\sigma_{\mathrm{vec}(\hat{J})} \, \sigma_{\mathrm{vec}(J)}}$$

Scalar Metrics

Let y_i denote the reference scalar value and $\hat{y}_i$ the predicted scalar value for run i, over N runs. $\bar{y}$ denotes the sample mean of the reference values.

$$\mathrm{MAE} = \frac{1}{N} \sum_{i=1}^{N} |\hat{y}_i - y_i|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (\hat{y}_i - y_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2}$$

Appendix F

Physics Diagnostic Definitions

Negative-flux fraction.
The fraction of predicted flux values that are negative across T time steps for a given run:

$$f_{\text{neg}} = \frac{1}{T}\sum_{n=1}^{T} \mathbf{1}\Big[\hat{J}(t_n) < 0\Big]$$

Since flux must be non-negative, any non-zero value indicates a physical violation in the surrogate output.

Bottom-boundary flux relative RMSE.
The normalised RMSE between the predicted flux curve and the reference flux curve at the lower boundary, measuring how closely the surrogate reproduces the boundary output:

$$\text{relRMSE}_{\text{bc}} = \frac{\sqrt{\dfrac{1}{T}\displaystyle\sum_{n=1}^{T}\big(\hat{J}(t_n) - J(t_n)\big)^2}}{\sqrt{\dfrac{1}{T}\displaystyle\sum_{n=1}^{T} J(t_n)^2} + \varepsilon}$$

Mass-balance relative RMSE.
Checks whether the surrogate flux output is consistent with conservation of drug mass in the domain. Using the true concentration snapshots as a reference, a residual r_n is computed at each time interval by comparing the observed rate of change of domain-mean concentration against the value implied by the predicted flux, the top-boundary inflow, and the reaction term. These residuals are then normalised by the magnitude of the observed concentration rate of change:

$$\text{relRMSE}_{\text{mass}} = \frac{\sqrt{\dfrac{1}{T-1}\displaystyle\sum_{n} r_n^2}}{\sqrt{\dfrac{1}{T-1}\displaystyle\sum_{n}\left(\frac{\Delta\bar{C}_n}{\Delta t_n}\right)^2} + \varepsilon}$$

A value near zero would indicate near-perfect mass conservation; a value of order one indicates the diagnostic is dominated by the numerical scale of the concentration dynamics rather than surrogate error.

Appendix G

1D Analytic Benchmark Derivation

This appendix derives the analytic solution used to verify the `V1` simulator. The problem is a one-dimensional slab of thickness L with drug held fixed at the top surface and removed instantly at the bottom. The solution is an exact formula for how concentration evolves over time, so that the numerical solver can be checked against it.

The governing equation is Fick's second law [2], which describes how concentration changes over time due to diffusion, with a constant diffusion coefficient D:

$$\frac{\partial C}{\partial t} = D\frac{\partial^2 C}{\partial z^2}.$$

Three conditions close the problem:

- $C(0,t) = C_0$ - the top surface is held at a fixed donor concentration
- $C(L,t) = 0$ - the bottom is held at zero, representing the perfect-sink boundary
- $C(z,0) = 0$ - the slab begins drug-free throughout

Step 1: Find the long-run solution. After a long time, the concentration stops changing and settles into a steady profile. At this point $\partial C/\partial t = 0$, which reduces the governing equation to:

$$\frac{\mathrm{d}^2 C}{\mathrm{d}z^2} = 0.$$

Integrating twice gives $C = az + b$ for some constants a and b, meaning the steady-state profile must be a straight line in z. The two boundary conditions $C(0) = C_0$ and $C(L) = 0$ fix a and b, giving:

$$C_{\mathrm{ss}}(z) = C_0\Big(1 - \frac{z}{L}\Big).$$

This is the linear profile the concentration tends towards as $t \to \infty$.

Step 2: Find the transient departure. At $t = 0$ the slab is entirely drug-free, so the concentration profile is a flat line at zero. The top boundary immediately jumps to C_0 but the rest of the slab takes time to fill in. Over time the profile curves upward from below, gradually straightening until it settles into the linear steady-state profile C_{ss}. The transient term u captures exactly this gap - it is the vertical distance between the current curved profile and the eventual straight line, which starts large and shrinks to zero as the slab fills in.

The solution is written as $C = C_{\mathrm{ss}} + u$, where u is this gap. Three things are known about u:

- **Initial value:** since the slab starts drug-free, $C(z,0) = 0$, so the initial gap is $u(z,0) = 0 - C_{\mathrm{ss}}(z) = -C_0(1 - z/L)$
- **Boundary values:** since C_{ss} already satisfies both boundary conditions exactly, u must be zero at $z = 0$ and $z = L$ for all time
- **Long-run behaviour:** as the system approaches steady state the gap closes, so $u \to 0$ as $t \to \infty$

Since u must be zero at both boundaries, it can be built from sine functions of the form $\sin(n\pi z/L)$, which are zero at $z = 0$ and $z = L$ for any positive integer n. Writing a single term as $u_n = \sin(n\pi z/L) \cdot T(t)$ and substituting into $\partial u/\partial t = D\,\partial^2 u/\partial z^2$ gives:

$$\sin\Big(\frac{n\pi z}{L}\Big)\,T'(t) = -\frac{n^2\pi^2 D}{L^2}\sin\Big(\frac{n\pi z}{L}\Big)\,T(t).$$

The sine term appears on both sides and cancels, reducing this to an equation in T alone:

$$T'(t) = -\frac{n^2\pi^2 D}{L^2}\,T(t) \quad \Longrightarrow \quad T(t) = \exp\!\left(-\frac{n^2\pi^2 D t}{L^2}\right).$$

The full solution for u is therefore a sum of these terms, each with its own amplitude B_n:

$$u(z,t) = \sum_{n=1}^{\infty} B_n \sin\Big(\frac{n\pi z}{L}\Big)\exp\!\left(-\frac{n^2\pi^2 D t}{L^2}\right).$$

Step 3: Find the coefficients. The amplitudes B_n are fixed by requiring $u(z,0) = -C_0(1 - z/L)$. Substituting $t = 0$ into the series gives:

$$\sum_{n=1}^{\infty} B_n \sin\Big(\frac{n\pi z}{L}\Big) = -C_0\Big(1 - \frac{z}{L}\Big).$$

To isolate a single B_n, both sides are multiplied by $\sin(m\pi z/L)$ and integrated over $[0, L]$. The sine functions are orthogonal, meaning:

$$\int_0^L \sin\Big(\frac{n\pi z}{L}\Big)\sin\Big(\frac{m\pi z}{L}\Big)\,\mathrm{d}z = \begin{cases} L/2 & n = m \\ 0 & n \neq m \end{cases}$$

so every term in the sum vanishes except the one where $n = m$, giving:

$$B_n = \frac{2}{L}\int_0^L -C_0\Big(1 - \frac{z}{L}\Big)\sin\Big(\frac{n\pi z}{L}\Big)\,\mathrm{d}z = -\frac{2C_0}{n\pi}.$$

Final result. Combining steady state and transient:

$$C(z,t) = C_0\Big(1 - \frac{z}{L}\Big) - \frac{2C_0}{\pi}\sum_{n=1}^{\infty}\frac{1}{n}\sin\Big(\frac{n\pi z}{L}\Big)\exp\!\left(-\frac{n^2\pi^2 D t}{L^2}\right).$$

This form follows directly from the separation-of-variables framework of Crank [2] applied to these specific boundary and initial conditions. In the benchmark implementation 200 series terms are retained, which is sufficient to reduce truncation error well below the discretisation error of the numerical scheme at all tested grid sizes.

Appendix H

Repository Layout

```
COMP3931_IndividualProject/
├── src/skin_diffusion/       # Core simulator, dataset, metrics, and ML utilities
│   ├── solver.py             #   Finite-difference transport update
│   ├── operators.py          #   Numerical operators
│   ├── bc.py                 #   Boundary-condition handling
│   ├── layers.py             #   Layer and diffusivity-field construction
│   ├── config.py             #   YAML configuration loading
│   ├── dataset.py            #   Run-bundle assembly logic
│   ├── dataset_spec.py       #   Dataset parameter sampling
│   ├── ml_dataset.py         #   ML feature/target export utilities
│   ├── ml_curve_backbone.py  #   Curve backbone fitting
│   ├── ml_metrics.py         #   ML curve metrics
│   └── ml_*diagnostics.py    #   Scalar and physics diagnostics
├── scripts/
│   ├── sim/                  # Simulation, validation, QC, and dataset scripts
│   └── ml/                   # Black-box and physics-corrected surrogate training
├── configs/sim/              # YAML simulation and dataset definitions
├── notebooks/                # Analysis and QC notebooks
├── docs/                     # Reproducibility and workflow documentation
├── tests/                    # Unit tests
├── results/                  # Curated final result artefacts
```

Appendix I

Dataset Quality Control Plots

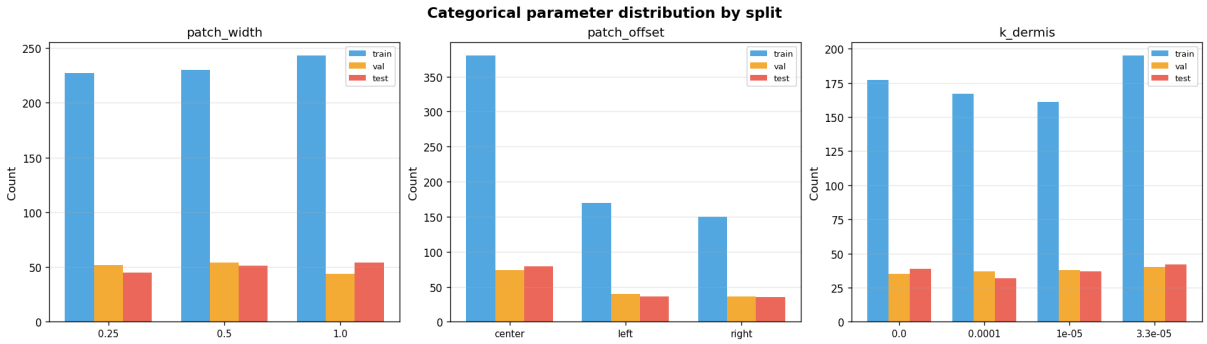


Figure I.1: Categorical parameter counts by split for `patch_width`, `patch_offset`, and `k_dermis`. Approximate balance across train, validation, and test confirms that the stratified partitioning preserves discrete design-variable coverage.

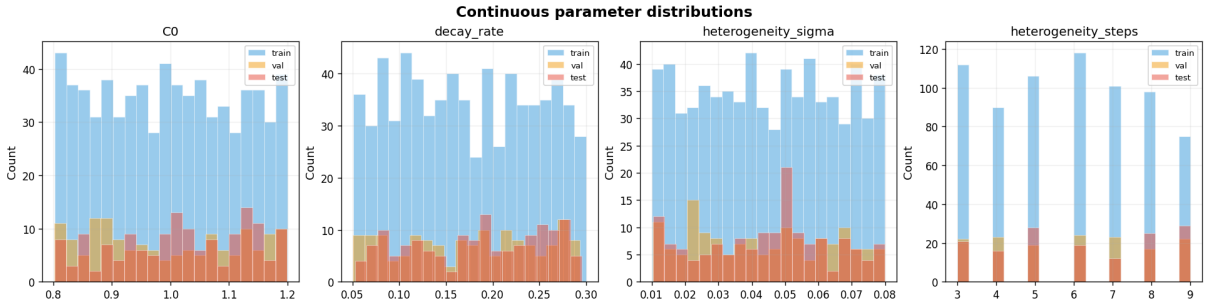


Figure I.2: Continuous parameter distributions by split for `C0`, `decay_rate`, `heterogeneity_sigma`, and `heterogeneity_steps`. Overlapping histograms across splits confirm that the sampled parameter space is consistently covered.

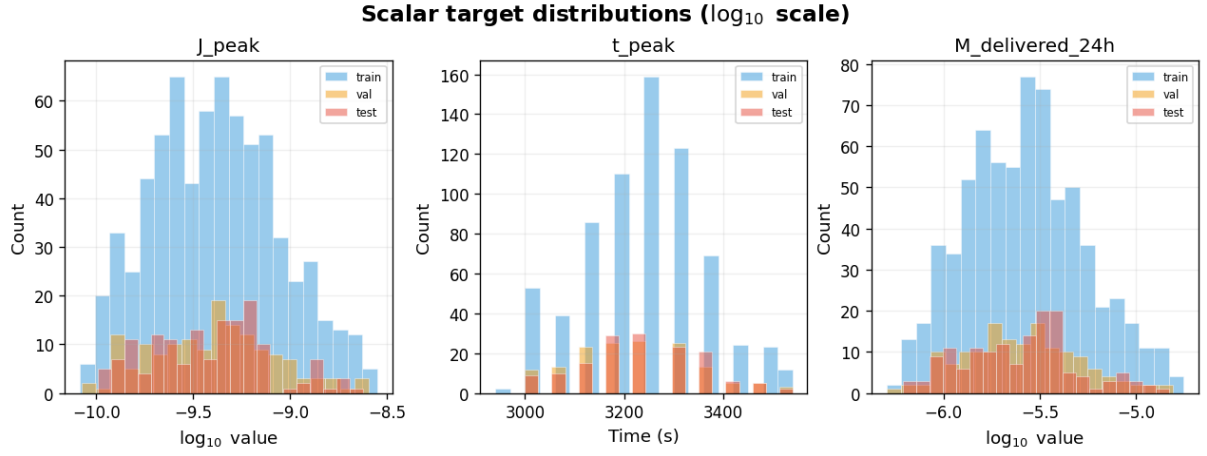


Figure I.3: Primary scalar target distributions for J_{peak}, t_{peak}, and $M_{\text{delivered},24h}$ by split. J_{peak} and $M_{\text{delivered},24h}$ are shown on a $\log_{10}$ scale; t_{peak} is shown on a linear scale.

Appendix J

Unit Test Modules

The test suite is organised into three groups covering the simulator kernel, dataset pipeline, and machine-learning components.

Simulator

`test_bc.py` Boundary-condition application: patch mask construction, Dirichlet and Neumann enforcement, and edge-patch handling.

`test_operators.py` Numerical diffusion operators: consistency between the constant-D and variable-D conservative finite-difference schemes.

`test_stability.py` Stability limit calculations: diffusion and reaction CFL conditions, including warning behaviour on violation.

`test_checks.py` Concentration field diagnostics: mass computation, L_2 error, and non-negativity assertions.

`test_metrics.py` Transport metric calculations: bottom flux, permeability, flux integration with cutoff, and finite-dose scalar quantities including peak timing and cumulative delivery.

Dataset Pipeline

`test_dataset_spec.py` Dataset parameter sampling: config injection and conditional sampling rules, including forced centre offset when patch width is full.

`test_dataset.py` Train/val/test index splitting: fraction validation, floating-point edge cases, and split coverage.

Machine Learning

`test_ml_dataset.py` ML dataset export: feature matrix assembly, interaction feature values, and target alignment across splits.

`test_ml_run_dataset.py` Run dataset loading: split index filtering, NPZ row alignment, and run bundle access for both flat and nested staging layouts.

`test_ml_scalar_diagnostics.py` Scalar diagnostics: safe R^2 computation for degenerate constant-target cases.

Appendix K

Scalar Transport Quantity Definitions

Three scalar summaries are extracted from the bottom-boundary flux curve $J(t)$ and used as surrogate regression targets.

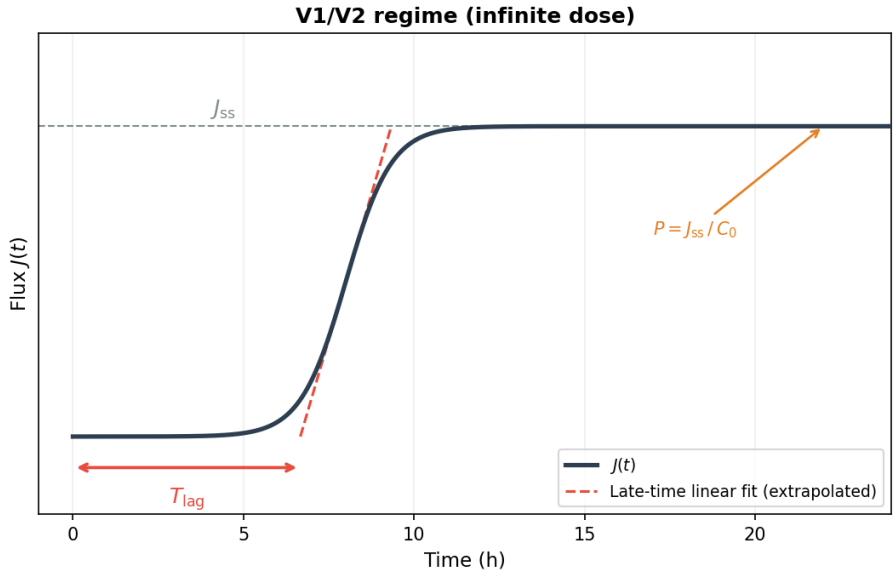


Figure K.1: V1/V2 (infinite dose): steady-state flux J_{ss}, lag time T_{lag} (x-intercept of the extrapolated linear fit), and permeability $P = J_{\mathrm{ss}}/C_0$.

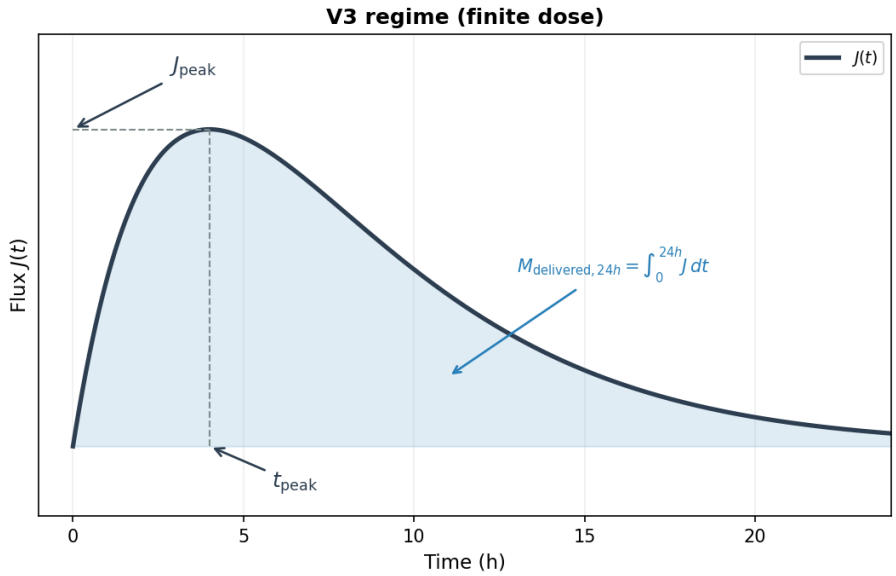


Figure K.2: V3 (finite dose): peak flux J_{peak}, time to peak t_{peak}, and 24 h delivered mass $M_{\mathrm{delivered,24h}} = \int_0^{24\mathrm{h}} J\,\mathrm{d}t$ (shaded area).

Appendix L

Hybrid Surrogate Training Convergence

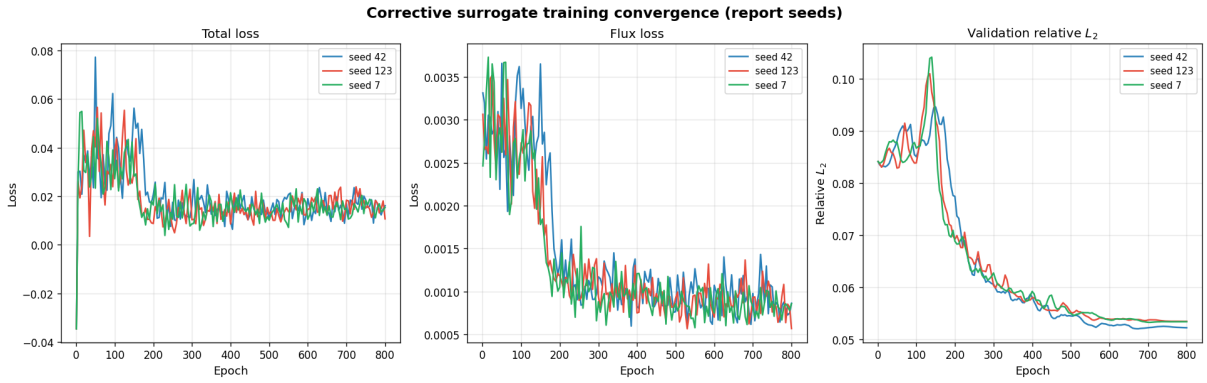


Figure L.1: stageCorrected training curves across seeds 42, 123, and 7: total loss (left), flux loss (centre), and validation relative L_2 (right). All three seeds show consistent convergence behaviour, with validation L_2 stabilising below 0.06 by approximately epoch 300. The early spike in total loss reflects the initial correction-scale warmup; no divergence was observed across any seed.

The consistent convergence across seeds supports the multi-seed stability findings reported in Chapter 4.